CS201 Spring 2026 — Lecture Notes

Clarissa Littler & Claude Opus

Spring 2026

Preface

This document is a synthetic running narrative of what we covered in each class meeting this term. Each section pairs the agenda for the day with the little code samples we actually typed up together, so if you missed a lecture or want to go back and re-read, the whole arc of a day should be here in one place. The source files themselves live in the `lecture`*N*`/` subdirectories of the class repo, and all of the snippets quoted below are verbatim from those files.

> Clarissa here: I read over these notes after generated and do some light editing but so far this seems to be a good workflow for preparing for post-facto notes based on the examples I show in class.

Contents

1 Lecture 1 — Welcome, C vs. C++, Bitwise Tricks, and a Peek at Integers

This one was mostly an orientation plus a crash-course in the parts of C that tend to trip up people coming from C++. I was genuinely a little surprised everyone showed up, so thank you for that!

Logistics

The class is my own personal admixture of four things:

- C
- x86-64 assembly
- Concurrency
- Operating systems theory and practice

Grading-wise, you've got quizzes, discussions, and coding assignments. The class repo is going to be your friend: that's where the textbook lives, where all the "Agenda" files like this one come from, and where we'll drop every bit of code we work on in class.

Structurally, I'll aim to start about five minutes after 10, break after every 50 minutes, and try not to keep you much past two hours.

C: What's different?

If you're coming from C++, the big shocks are:

- No pass-by-reference. You get pass-by-value or you get pointers, full stop.
- No `bool` type (well, not in the way you're used to — nonzero is truthy and zero is falsy, and there's no native `true`/`false`).
- No `nullptr`; you use the `NULL` macro (which is basically just `0` cast to a pointer).
- I/O is `printf`/`scanf` rather than streams.

We played with two small files to make this concrete. The first, `arraything.c`, shows that arrays do decay to pointers — so you pass the length by hand — and that you iterate with a plain `for` loop rather than an iterator:

```
void arrayMunger(int a[], int s){
  for(int i=0; i<s; i++){
    a[i] = 2*a[i];
  }
}
```

The little commentary `// arr[i] ===> *(arr + i)` hints at the thing we'll lean on all term: indexing is just pointer arithmetic in disguise.

The second file, `passPointy.c`, makes the "no pass by reference" point tangible. If you want a function to modify its caller's variable, you pass the *address* and dereference inside:

```
void intMunger(int* p){
  *p = 2*(*p);
}
```

And you call it with `intMunger(&test);`. That ampersand is doing real work — it's giving you a pointer to `test` rather than a copy of it.

Bitwise operations

We went through the usual suspects with truth tables:

- `&` (AND), `|` (OR), `^` (XOR), `~` (NOT).
- `<<` (shift left, pads with zeroes, anything that falls off the edge is gone) and `>>` (shift right, same idea on the other side).

The file `bitmunger1.c` is where this all came together. It defines a little loop that prints out the 32 bits of an `int`, lets you pick a bit to toggle, and optionally negates the integer via two's complement:

```
void toggleBit(int* num,int place){
  *num = (*num) ^ (1 << place);
}

void twosComplement(int* num){
    *num = ~((*num) - 1);
}
```

`toggleBit` is the canonical XOR trick: XOR-ing a bit with 1 flips it, XOR-ing with 0 leaves it alone. `twosComplement` implements the "subtract one, then invert" rule for negating a two's-complement integer.

Peeking inside an integer: two's complement

We derived why two's complement works. A plain unsigned n-bit integer encodes

$$\sum_{i=0}^{n-1} b_i \cdot 2^i,$$

so `11111111` (8 bits) is $128 + 64 + \cdots + 1 = 255$. In two's complement, the top bit's weight flips sign:

$$-b_{n-1} \cdot 2^{n-1} + \sum_{i=0}^{n-2} b_i \cdot 2^i.$$

That gives you one clean mental check: all-ones is always -1, because $-2^{n-1} + (2^{n-1} - 1) = -1$. And to negate a number, you subtract one then invert all the bits (or equivalently, invert and then add one).

A preview of floats

We ran out of time before we could fully explore floats, but I showed off `floatmunger1.c`, which is structurally the same as `bitmunger1.c` but reinterprets an `int`'s bits as a `float` via a pointer cast. The little spaces it prints (between bits 31/30 and 23/22) are hinting at the IEEE-754 sign / exponent / fraction split we'll dig into next time.

2 Lecture 2 — Hex, Endianness, IEEE-754, and Where Floats Lie to You

The plan was to finish the float discussion we previewed on day one, but before diving in we took a small detour through hex and endianness.

Warm-up: hex codes and the shape of a pointer

Hex is base-16, digits 0–F. The reason it shows up everywhere — addresses, colors, byte dumps — is that two hex digits are exactly one byte ($16 \cdot 16 = 256 = 2^8$). So when you see an RGBA color like `#8765AF`, that's really three (or four) bytes: one for red, one for green, one for blue, optionally one for transparency.

Endianness

We wrote `byteOrderTest.c` to actually look at how an 8-byte integer is laid out in memory on our machines:

```
long int num = 0x0123456789abcdef;
unsigned char* p = (unsigned char*)&num;

printf("The bytes are stored in memory as: ");
for(int i=0; i<8; i++){
  printf("%x ",*(p+i));
}
```

On an x86-64 box this prints the bytes out in the opposite order from how we wrote the literal — classic little-endian. The trick is the `(unsigned char*)` cast: once we have a byte-pointer, pointer arithmetic moves us one byte at a time, so we can walk the actual storage.

Sizes and pointers

Alongside that, `sizes.c` just prints `sizeof(int)`, `sizeof(long)`, `sizeof(char)`, `sizeof(float)`, `sizeof(double)`, and the size of a pointer. On our 64-bit Linux boxes: int is 4, long is 8, char is 1, float is 4, double is 8, and any pointer is 8. That last one is the important bit — *all* pointers are the same size, regardless of what they point to, because they're all just addresses.

And `pointerArith.c` drove home that pointer arithmetic scales with the *pointee* size. An `int*` advances 4 bytes per `+1`, a `char*` advances 1, a `char**` advances 8, and so on.

IEEE-754, formally

We did the 32-bit (`float`) version because the 64-bit version is just "same thing with bigger fields."

- 1 bit sign
- 8 bits exponent (stored with a bias of 127)
- 23 bits fraction

In the normal case, the value is

$$(-1)^s \cdot 2^{e-127} \cdot (1+f),$$

where f is the fractional part read as "rational binary": the bit at position k after the implicit point contributes 2^{-k}.

Worked example: `0 10000001 1010...0` is

$$(-1)^0 \cdot 2^{129-127} \cdot (1+\tfrac{1}{2}+\tfrac{1}{8}) = 4 \cdot \tfrac{13}{8} = 6.5.$$

We exercised this live with `floatmunger1.c` (same idea as lecture 1, but now with `%.16f` so you can see lots of digits). Flipping specific bits and watching the printed value change is the single best way to internalize the format.

Edge cases

- Exponent is all 1s:
  - fraction is zero $\Rightarrow \pm\infty$,
  - fraction is nonzero $\Rightarrow$ `NaN`.
- Exponent is all 0s (subnormal numbers): the exponent sticks at 2^{-126}, but the leading implicit 1 is gone, so the value is $(-1)^s \cdot 2^{-126} \cdot f$.

Where floats lie to you

`floatlimit.c` is the "gotcha" example:

```
float f = 0.0001;
float accum = 0;
for(int i=0; i<10000; i++){
  accum += f;
}
printf("%.20f\n",accum);
```

We'd love this to print `1.00000000000000000000`. It doesn't. The reason is a mix of: 0.0001 is not exactly representable in binary, and we're accumulating that small rounding error 10,000 times.

Why is this *unavoidable*? A bit of math: the rationals and the integers are countably infinite, but the reals form a strictly larger uncountable set. There is no way to cover the reals with finitely many bits, or even with countably many bits; we're forced to pick a countable scatter of "breadcrumbs" and round every other real to the nearest one.

Round to even

To keep that rounding error from biasing one direction, IEEE-754 specifies "round to even": when a real is exactly halfway between two breadcrumbs, the hardware rounds to the one whose last bit is zero. Over many operations this rounds up about as often as it rounds down.

3 Lecture 3 — `malloc`, and Your First Real Assembly

The real headline today: we left C-land and wrote actual assembly against the bare Linux syscall interface. But first, I owed you `malloc`.

`malloc`, finally

In my rush last time I completely skipped dynamic allocation, which is embarrassing, so we opened with two quick examples.

`malloc.c` is the minimum viable pointer dance:

```
int* n = malloc(sizeof(int));
*n = 10;
printf("%d\n",*n);
free(n);
n = 0;
if(0 == NULL){
  printf("Yup\n");
}
```

Two things to note. First, `malloc` always returns a pointer and always needs you to tell it how many bytes you want — unlike `new T` in C++, it doesn't know anything about your type, so you pass `sizeof(int)` (or a multiple of it). Second, that final `if` is my excuse to point out that `NULL` is literally just `0` cast to a pointer type.

`malloc2.c` extends this to an array and a struct:

```
int* narr = malloc(100*sizeof(int));
narr[2] = 5;

struct Goofus* g = malloc(sizeof(struct Goofus));
g->g1 = 5; // same thing as (*g).g1
```

Once you've got a pointer from `malloc`, you can index it just like an array. The `->` arrow is shorthand for "dereference then access a field."

Stepping back: what is a CPU, anyway?

The big conceptual pivot for the rest of the term:

- A CPU is, at bottom, a thing that fetches *instructions* from memory and does what they say.
- The set of instructions it understands is an *instruction set architecture* (ISA). Ours is x86-64.
- Assembly language is a one-to-one (ish) textual encoding of those instructions, so writing assembly is as close to "talking to the CPU" as we'll practically get.
- Registers are a tiny amount of extremely fast storage inside the CPU itself. Most instructions operate on registers; memory comes in via load/store.
- Memory hierarchy, the 30-second version: registers → L1/L2/L3 cache → main memory → disk. Each level is roughly an order of magnitude slower and roughly an order of magnitude bigger than the one above it.

I also mentioned `gcc -S`, which tells gcc to stop at assembly rather than assemble and link. We'll come back to it.

Your first bare assembly program

`first.s` is deliberately broken:

```
    .section .text
    .global _start

_start:
    mov $60,%rax
```

This is the shortest thing that looks like an assembly program. The `.text` section is where code lives; `_start` is the symbol the linker uses as the program entry point (unlike in a C program, there is no runtime to call `main` for us).

We moved `60` into `%rax` — 60 is the syscall number for `exit` on x86-64 Linux — and then... did nothing else. Because we never actually issued the `syscall` instruction and there's no implicit return, the CPU just wandered off the end of our code and trapped. Hence the crash.

`firstfixed.s` is the corrected version:

```
_start:
    mov $60,%rax
    mov $-1,%rdi
    syscall
```

Syscall convention: put the syscall number in `%rax`, put arguments in `%rdi`, `%rsi`, `%rdx`, etc., and then execute `syscall`. For `exit(status)`, the status goes in `%rdi`. So this exits with status `-1`, and `echo $?` in the shell will confirm it.

`add` and `sub`

`firstadd.s` and `firstsub.s` are our first binary operations:

```
_start:
    mov $10,%rbx
    mov $20,%rcx
    add %rbx,%rcx
    mov %rcx,%rdi
    mov $60,%rax
    syscall
```

Syntax note that will bite you all term: in AT&T syntax, `add S, D` means `D = D + S`. The destination is on the *right*. So `add %rbx,%rcx` leaves 30 in `%rcx`. `sub` is analogous: `sub %rbx,%rcx` computes `D - S` and leaves 10 in `%rcx`.

We also did the fun thing of running `gcc -S test.c` on a tiny C file and skimming the output (`test.s`). It's noisy (`.cfi_`directives, frame setup, PLT calls) but the actual work is recognizable: move 10 into a local, load it into `%esi`, load the format string's address via `leaq`, call `printf@PLT`.

4 Lecture 4 — Registers, Calling Convention, Addressing Modes, Control Flow, and "Hello, world"

Today was a dense one: we filled in the register table, introduced the Linux x86-64 calling convention, walked through every addressing mode we'll use for arrays, covered `cmp`/`jmp`, wrote a for-loop in assembly, and finished by doing `write` and `exit` syscalls by hand.

Logistics

The Linux server admin is in the process of activating accounts for folks who can't log in yet, so hang tight if that's you.

Register reference

The basic x86-64 general-purpose registers and what they were historically used for:

- `%rax` — accumulator (return values, implicit operand for `mul`/`div`).
- `%rbx` — base register (historically an array base).
- `%rcx` — counter (loops, `rep`).
- `%rdx` — data (I/O, upper half of multiplication/division).
- `%rsi` — source index (string ops).
- `%rdi` — destination index (string ops).
- `%rbp` — base pointer (stack frame base).
- `%rsp` — stack pointer (top of the stack).
- `%r8`–`%r15` — extra general-purpose registers.
- `%rip` — instruction pointer; we don't write to this directly, but we *do* use it in RIP-relative addressing.

The floating-point/SIMD registers (`%xmm0`...) live in the `assemblyGuide.org` in the repo.

Linux/x86-64 calling convention, first pass

- Return values go in `%rax`.
- The first six integer/pointer arguments are passed in, in order: `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8`, `%r9`.

Syscalls use a slightly different convention (the fourth argument is `%r10`, not `%rcx`), but for function calls in user code the list above is what you want.

`add`/`sub`, again

We re-ran `firstadd.s` from last time as a warm-up; the only new thing is a pair of clarifying comments explaining that `%rdi` is the exit-code argument and `60` in `%rax` selects the `exit` syscall.

Global variables: values, pointers, and `lea`

Three files building on each other:

`addGlobal.s` — define a global with `.quad`, then load, add, store, load-again:

```
    .section .data
num: .quad 200

_start:
    mov num,%rbx # load num into %rbx
    add $10,%rbx
    mov %rbx,num # store back
    mov num,%rdi
    mov $60,%rax
    syscall
```

This works, but referencing a symbol like `num` directly makes the assembler emit an absolute address, which is awkward for position-independent code.

`addGlobalBetter.s` — same thing, but using *RIP-relative* addressing:

```
    mov num(%rip),%rbx
    add $10,%rbx
    mov %rbx,num(%rip)
```

Here `num(%rip)` means "the address that is `num`-minus-the-next-instruction bytes away from the current instruction pointer." The assembler computes that offset for us at build time. This is the form you want in modern code.

`addGlobalLea.s` — now using `lea` to take *the address of* `num`, the way `&` does in C:

```
    lea num(%rip),%rbx # rbx = &num
    addq $10,(%rbx) # *rbx = *rbx + 10
    mov num(%rip),%rdi
    mov $60,%rax
    syscall
```

The comment block at the bottom of the file spells out the C equivalent:

```
int num = 200;
int* nump = &num;
*nump = *nump + 10;
```

Parentheses around a register mean "dereference." So `(%rbx)` is "the 8 bytes of memory starting at the address held in `%rbx`."

From a scalar to an array

Four files, one new addressing idea at a time.

`addArray1.s` — change `num` to an array of four quadwords. The code is unchanged, so it just touches the first element.

`addArray2.s` — to get at element 1, add 8 to the pointer (because one `.quad` is 8 bytes):

```
    lea num(%rip),%rbx
    addq $8,%rbx
    addq $10,(%rbx)
    movq (%rbx),%rdi
```

`addArray3.s` — same effect, but using the full "scale-index" addressing mode:

```
    lea num(%rip),%rbx
    mov $1,%rcx
    addq $10,(%rbx,%rcx,8) # *(base + index*8)
    movq (%rbx,%rcx,8),%rdi
```

The syntax `(base, index, scale)` computes `base + index*scale`. The scale has to be 1, 2, 4, or 8. This is exactly how C-style array indexing compiles — you'll see the same form over and over.

`addArray4.s` — same as `addArray2`, but with `.long` (4-byte) elements instead of `.quad`, which forces us to use the `l` suffix (`addl`, `movl`) and the 32-bit register name `%edi`.

Note the subtle point: with `.long`, a step of "one element" is 4 bytes, not 8. In the file as written, we advanced the pointer by 8 bytes, which lands us at element *two* of the `.long` array, not element one. That kind of size/stride mismatch is a classic bug I'll keep harping on all term.

`cmp` and `jmp`

`cmp S, D` does (roughly) `D - S` and sets flags without saving the result anywhere. The conditional jumps then read those flags:

- `jmp` — unconditional.
- `jl` — $D < S$. (Yes, `cmp S, D` then "if less" compares D to S; it still surprises me.)
- `jle` — $D \leq S$.
- `je`, `jne` — equal, not equal.
- `jg`, `jge` — greater, greater-or-equal.

The demo, `cmp1.s`:

```
    mov $10,%rbx
    mov $20,%rcx
    mov $-1,%rdi
    cmp %rbx,%rcx # flags set as if computing %rcx - %rbx
    jge greater
    mov $1,%rdi
greater:
    mov $60,%rax
    syscall
```

Since $20 \geq 10$, we jump to `greater` and exit with status `-1` (or `255` as far as the shell is concerned).

A for-loop in assembly

Two flavors of "sum the numbers from 1 to 10."

`sumLoop.s` counts up:

```
    mov $0,%rbx # accumulator
    mov $1,%rcx # counter
loopStart:
    add %rcx,%rbx
    add $1,%rcx
    cmp $10,%rcx
    jle loopStart
```

`sumLoopB.s` counts down:

```
    mov $0,%rbx
    mov $10,%rcx
loopStart:
    add %rcx,%rbx
    sub $1,%rcx
    cmp $0,%rcx
    jg loopStart
```

Both leave 55 in `%rbx`, which we then move into `%rdi` so we can read it out as the exit status.

Buffers, strings, and `write`

The grand finale, `hello.s`:

```
    .section .data
msg: .asciz "Hello world!\n"
hellolen = . - msg

    .section .text
    .globl _start

_start:
    mov $1,%rax # syscall number for write
    mov $1,%rdi # fd = 1 (stdout)
    lea msg(%rip),%rsi # buffer address
    mov $hellolen,%rdx # number of bytes
    syscall

    xor %rdi,%rdi # exit status 0
    mov $60,%rax # syscall number for exit
    syscall
```

Points of interest:

- `.asciz` lays down the characters followed by a null terminator.
- `hellolen = . - msg` is a classic assembly idiom: the dot is "current address," so subtracting the start of `msg` gives the length in bytes, including the newline and the terminator. We use that as the third argument to `write`.

- `write` is syscall 1 on Linux x86-64. Arguments are `(fd, buf, count)` in `%rdi`, `%rsi`, `%rdx`.
- `xor %rdi,%rdi` is the idiomatic way to zero a register — one byte shorter than `mov $0,%rdi`.

Next time we'll close this out with an `echo`-style program that *reads* from stdin and writes back, which means handling buffers properly.

5 Lecture 5 — Hello Again, `echo`, and the Road from Jumps to Proper Functions

Today was the big one for putting structure around assembly: we re-did "hello, world" carefully, wrote an `echo` that reads and writes a buffer, then walked the whole arc from `jmp`-style subroutines to `call`/`ret` and finally to real stack frames with local variables. By the end, we had a function in assembly that looks and behaves like a function in C.

Hello, world — this time with feeling

We re-examined `hello.s` slowly, focusing on the pieces we glossed over last time:

```
    .section .data
msg: .asciz "Hello world!\n"
hellolen = . - msg

    .section .text
    .global _start

_start:
    mov $1,%rax # write syscall
    mov $1,%rdi # fd = 1 (stdout)
    lea msg(%rip),%rsi # buffer
    mov $hellolen,%rdx # length
    syscall

    xor %rdi,%rdi
    mov $60,%rax
    syscall
```

A few things worth restating:

- File descriptors in Unix are just integers indexing into a per-process table the kernel maintains. `0` is stdin, `1` is stdout, `2` is stderr, and anything you `open` returns the next free slot. `write` doesn't care what kind of thing the fd refers to — pipe, socket, file, terminal — it all looks the same from up here.
- The `write` syscall *returns* something in `%rax`: the number of bytes it actually wrote (or a negative errno on failure). We didn't use it here, but we will.

`helloRet.s` is the tiny variant that does use it:

```
    mov $1,%rax
    mov $1,%rdi
    lea msg(%rip),%rsi
    mov $hellolen,%rdx
```

```
    syscall

    mov %rax,%rdi # return value of write becomes our exit status
    mov $60,%rax
    syscall
```

Run it and then `echo $?` — you'll see the byte count of "Hello world!\n" come out as the exit status. This is the idea I want you to carry forward: `%rax` holds the return value of whatever you just called (syscall or function), and you can feed it into whatever comes next.

Reading input: `.bss` and the `read` syscall

To read we need somewhere to read *into*, which is what the `.bss` section is for — uninitialized, zero-filled storage that costs nothing on disk because the loader knows to just hand us zeroed pages:

```
    .section .bss
buff: .skip 128
```

`.skip 128` reserves 128 bytes at the label `buff`.

`read` is syscall 0, and its arguments mirror `write`:

- `%rdi`: fd (0 = stdin)
- `%rsi`: pointer to the buffer
- `%rdx`: maximum number of bytes

And just like `write`, `read` returns the number of bytes actually read in `%rax`.

`echo.s`: read and then write

Here's `echo.s` end-to-end:

```
    .section .bss
buff: .skip 128

    .section .text
    .global _start

_start:
    mov $0,%rax # read
    mov $0,%rdi # stdin
    lea buff(%rip),%rsi
    mov $128,%rdx
    syscall

    mov %rax,%rdx # bytes-read becomes the count for write
    mov $1,%rax # write
    lea buff(%rip),%rsi
    mov $1,%rdi # stdout
    syscall

    xor %rdi,%rdi
    mov $60,%rax
```

```
    syscall
```

The clever step is the first line after `syscall`: `mov %rax,%rdx`. `read` told us how much it got; we immediately funnel that into the byte-count argument of `write`, so we only echo back what was actually typed. If we hardcoded 128 there instead, we'd dump the uninitialized tail of the buffer too — not a crash, but garbage.

Jump-based "subroutines" — and why they don't scale

Before `call`/`ret` existed conceptually for us, the hack was to use labels and unconditional jumps. `routine.s` is that hack:

```
rdadder:
    add %rdi,%rdi
    mov %rdi,%rax
    jmp rdret

_start:
    mov $20,%rdi
    jmp rdadder
rdret:
    mov %rax,%rdi
    mov $60,%rax
    syscall
```

It works: we jump to `rdadder`, it doubles `%rdi`, then `jmp rdret` gets us back. But look at what we had to do — the "return address" is hardcoded into the function body. If we wanted to call `rdadder` from two different places, we'd have to either duplicate it or add some awful switch. It's not reusable, it's not encapsulated, it's not a function.

Real functions: `call` and `ret`

`call` and `ret` are two special instructions that fix the whole thing. Conceptually:

- `call label` pushes the address of the *next* instruction onto the stack and then jumps to `label`.
- `ret` pops that address off the stack and jumps to it.

So `call`/`ret` are just "save your place, jump, come back" — and because the save uses the stack, you can call as many nested functions as you want.

`fun1.s` is the first real function we wrote:

```
rdadder:
    add %rdi,%rdi
    mov %rdi,%rax
    ret

_start:
    mov $20,%rdi
    call rdadder # %rax = 40
    mov %rax,%rdi
    call rdadder # %rax = 80
```

```
    mov %rax,%rdi
    mov $60,%rax
    syscall
```

Now `rdadder` is reusable — we call it twice in a row and compound the doubling. Exit status should be 80.

`fun2.s` is a variant that uses `%r9` as a scratch register instead of clobbering `%rdi`:

```
rdadder:
    mov %rdi,%r9
    add %r9,%r9
    mov %r9,%rax
    ret

_start:
    mov $20,%rdi
    call rdadder # %rdi still 20, %rax = 40
    call rdadder # %rdi still 20, %rax = 40 again
    mov %rax,%rdi
    mov $60,%rax
    syscall
```

Notice what's different from `fun1.s`: the `_start` here calls `rdadder` twice back-to-back without reloading `%rdi` in between. Because `rdadder` no longer modifies `%rdi` (that's the point of using `%r9` as a scratch), both calls see the same input and produce the same output. Exit status is 40, not 80. If you want to compound the effect, you have to feed the previous result back in as the next argument — exactly what `fun1.s` does with its `mov %rax,%rdi` between calls. The lesson: "the function is pure" and "calling it twice doubles the answer" are two very different statements.

Local variables mean we need a stack frame

For anything more interesting than "double a number," functions need scratch space — local variables. Every process gets a big region of memory called the stack, and every function call gets a contiguous slice of it called a *stack frame*.

Two registers run the show:

- `%rsp` — stack pointer, points at the "top" of the stack.
- `%rbp` — base pointer, points at the "bottom" of the current frame.

Important weirdness: the stack *grows downward*. "Top" is the lowest address. Pushing onto the stack *decreases* `%rsp`; popping *increases* it. A function's local variables live at negative offsets from `%rbp` (e.g. `-8(%rbp)`, `-16(%rbp)`).

`push` and `pop` are the two primitives: `push %rbx` subtracts 8 from `%rsp` and stores `%rbx` there; `pop %rbx` does the inverse.

The canonical function prologue:

1. `push %rbp` — save the caller's base pointer.
2. `mov %rsp,%rbp` — make our new frame start where the stack currently is.

3. `sub $N,%rsp` — carve out N bytes of locals.

And the epilogue undoes it:

1. `mov %rbp,%rsp` — drop all the locals in one move.
2. `pop %rbp` — restore the caller's base pointer.
3. `ret`.

`funStack.s`: a function with locals, by hand

We're computing $f(x) = 20 + 2x$, but deliberately using local variables for both `a = 20` and the intermediate $2x$:

```
fun:
    push %rbp
    mov %rsp,%rbp
    sub $16,%rsp # room for two quadword locals

    mov %rdi,-8(%rbp) # local1 = x
    add %rdi,-8(%rbp) # local1 += x (so local1 = 2x)
    movq $20,-16(%rbp) # local2 = 20
    movq -16(%rbp),%rax
    add -8(%rbp),%rax # %rax = 20 + 2x

    mov %rbp,%rsp
    pop %rbp
    ret

_start:
    mov $10,%rdi
    call fun
    mov %rax,%rdi
    mov $60,%rax
    syscall
```

With input 10, this exits with status 40. Walk through it once on paper: draw a stack growing downward, watch `push %rbp`, `mov %rsp,%rbp`, and `sub $16,%rsp` carve out the frame. The `-8(%rbp)` and `-16(%rbp)` slots are *your* two local variables; naming them in a comment alongside the code is a lifesaver when this gets bigger.

One subtle detail we'll formalize later: after `push %rbp` we've pushed one 8-byte value, which actually re-aligns `%rsp` to a 16-byte boundary (assuming the caller's `call` left it 8-aligned, which it does). That matters because the ABI requires `%rsp` to be 16-aligned at the point of a `call`, and some instructions (notably SSE moves) fault if it isn't.

6 Lecture 6 — Reusing Functions Across Files, Recursion, and Summing an Array

With `call`/`ret` and stack frames in hand, today was about doing things that actually resemble programming: linking pre-written `readInt`/`writeInt` helpers in with our code, writing a recursive `factorial`, and walking an array to sum it up.

Logistics

First, grades for the stack assignments are in — nice work overall.

Second, the next coding assignment: write an assembly program that *(i)* declares an array, *(ii)* reads input, *(iii)* does something interesting combining the two, and *(iv)* exits cleanly. Two constraints: you must assemble with `as` and link with `ld` (no `gcc` driver), and you're encouraged to pull in the `readInt`/`writeInt` helpers from the assembly guide. Due May 10th.

Third, Discussion 3 is on *posits*, an alternative to IEEE-754. Read `positTutorial.org` in the repo. The short pitch: posits spread their precision more evenly across the number line, so they give up some resolution at very small magnitudes in exchange for much better resolution at moderate ones, and they ditch the menagerie of special cases (no $\pm\infty$, no `NaN`, just a single `NaR`). A posit is made up of

- a sign bit,
- *regime* bits (a kind of super-exponent, variable width),
- exponent bits (variable width),
- fraction bits (whatever's left).

The variable widths are what let posits reallocate precision dynamically. We'll leave the details for the tutorial.

Linking in functions from other files

Up to now, everything we've built has been a single `.s` file with a `_start`. Real programs are many files linked together. To use a function defined elsewhere you just need to tell the assembler "this symbol is defined in another file"; the linker will wire it up later.

`readInt.s` defines (and makes global with `.global`) two functions:

```
    .section .bss
buffer: .zero 128

    .section .text
    .global readInt

parseInt:
    movzbl (%rdi,%rcx,1),%r8d # read one byte, zero-extended
    sub $'0',%r8
    imul $10,%rax
    add %r8,%rax
    inc %rcx
    cmp %rsi,%rcx
    jb parseInt
    ret

readInt:
    push %rbx
    mov $0,%rax # sys_read
    mov $0,%rdi # stdin
    lea buffer(%rip),%rsi
    mov $128,%rdx
```

```
    syscall

    mov %rax,%rbx # bytes read
    mov $0,%rcx
    mov $0,%rax
    lea buffer(%rip),%rdi
    dec %rbx # trim trailing newline
    mov %rbx,%rsi
    call parseInt
    pop %rbx
    ret
```

Two details worth pointing out. First, `movzbl` ("move byte to long, zero-extended") loads a single byte of the input buffer into the low 8 bits of `%r8` and clears the upper bits. Without the zero extension, garbage from whatever used `%r8` last would corrupt your arithmetic — classic bug. Second, `readInt` does `push %rbx` / `pop %rbx` around its body because `%rbx` is *callee-saved* in the Linux ABI. If we clobbered it, the caller would be entitled to complain.

`writeInt.s` is the mirror image: it takes an integer in `%rdi`, converts it to decimal digits by repeatedly dividing by 10, and prints them with `write`. A few lessons buried in there:

- It allocates a 32-byte buffer *on the stack* and fills it *from the end* — that's how "print digits in reverse order" naturally handles being emitted left-to-right later.
- It handles negatives by remembering a flag and negating `%rax`, with a special case for `INT_MIN` (-2^{63}), which can't be negated in two's complement. The code uses `movabsq` (not plain `movq`) because the constant 2^{63} doesn't fit in a 32-bit sign-extended immediate.
- It preserves `%rbx` (callee-saved again), and uses `leave` as shorthand for "`mov %rbp,%rsp` then `pop %rbp`."

To actually call these from another file, we use `.extern` (a declaration, not a definition):

```
    .section .text
    .global _start
    .extern readInt
    .extern writeInt

_start:
    call readInt
    mov %rax,%rdi
    call writeInt

    mov $60,%rax
    xor %rdi,%rdi
    syscall
```

That's `readWriteTest.s`. To build it you assemble each file separately with `as` and then link them all with `ld`:

```
as -o readWriteTest.o readWriteTest.s
as -o readInt.o readInt.s
as -o writeInt.o writeInt.s
ld -o readWriteTest readWriteTest.o readInt.o writeInt.o
```

Type a number, hit enter, and it prints the same number back. Not very exciting on its own, but it's the scaffolding for everything else today.

Recursion: factorial in assembly

`fact.s` is the first recursive function we've written. The idea is familiar — $n! = n \cdot (n-1)!$, with base case $0! = 1! = 1$ — but implementing it in assembly forces you to be explicit about how the stack carries your intermediate state.

```
fact:
    push %rbp
    mov %rsp,%rbp
    sub $16,%rsp # one local + padding for alignment

    cmp $1,%rdi
    jle basecase

    mov %rdi,-8(%rbp) # save n before the recursive call
    dec %rdi # %rdi = n - 1
    call fact # recursive call: returns (n-1)! in %rax
    mov -8(%rbp),%rdi # reload our saved n
    imul %rdi,%rax # %rax = n * (n-1)!
    jmp leavefact

basecase:
    mov $1,%rax

leavefact:
    leave
    ret
```

The essential move is `mov %rdi,-8(%rbp)` before the recursive call. `%rdi` is *caller-saved* — any function you call is allowed to clobber it — and recursion means we need `n` again after `fact(n-1)` returns. So we park it in our stack frame. Each recursive activation gets its own `-8(%rbp)` slot, because each activation has its own `%rbp`. That's the whole magic trick.

The `sub $16,%rsp` is worth pausing on. We only need 8 bytes for our one local, but we carve out 16 to keep `%rsp` 16-byte aligned at the next `call`. If this ever drifts you'll see mysterious crashes inside library functions that assume SSE alignment.

The `_start` that drives it:

```
_start:
    mov $1,%rax # print "Enter a number: "
    mov $1,%rdi
    lea message(%rip),%rsi
    mov $16,%rdx
    syscall

    call readInt
    mov %rax,%rdi
    call fact
    mov %rax,%rdi
    call writeInt
```

```
    mov $60,%rax
    xor %rdi,%rdi
    syscall
```

Try 5 — you should get 120. Try 20 — 2432902008176640000. Try 21 — you'll overflow a 64-bit signed integer and get nonsense. That silent wraparound is worth remembering.

Summing an array

`sumLoopArray.s` pulls together the array addressing from lecture 4, the counter-loop idiom, and `writeInt`:

```
    .section .data
arr: .quad 1,2,3,4

    .section .text
    .global _start
    .extern writeInt

_start:
    mov $0,%rcx # i = 0
    mov $0,%rax # accumulator
    lea arr(%rip),%r15 # base pointer

compare:
    cmp $4,%rcx
    jge done
loop:
    add (%r15,%rcx,8),%rax # %rax += arr[i]
    inc %rcx
    jmp compare
done:
    mov %rax,%rdi
    call writeInt

    mov $60,%rax
    xor %rdi,%rdi
    syscall
```

The key line is `add (%r15,%rcx,8),%rax`: scale-index addressing again, with scale 8 because we're stepping through `.quad` elements. If you change `arr` to `.long` you must change the scale to 4 *and* switch to the 32-bit forms (`addl`, `%eax`) — this is the same size/stride trap I warned about in lecture 4, and it will bite people in the assignment.

Output for this program is `10`, because $1 + 2 + 3 + 4 = 10$. If you wanted to sum a user-supplied run of numbers instead of a hardcoded array, you already have all the pieces: `readInt` in a loop, accumulate in `%rax`, `writeInt` at the end. That's a good shape for the upcoming assignment.

7 Lecture 7 — Processes, `fork`, Threads, and Why Sharing State Hurts

We're pivoting away from assembly for a bit and back into C, but with a new lens: *concurrency*. Today's arc was processes vs. threads, `fork` from the bottom up, a quick detour through function pointers because the threading API needs them, and then a first taste of why shared state between threads is a minefield.

Logistics

A few schedule notes worth surfacing: this term, "finals week" lands on what would have been week 10, so I've shifted lectures to fit in nine weeks. There's no in-class final — the last week is buffer time to wrap up the big assignments. Wednesday night of finals week is the hard cutoff for late homework (concretely: before I sit down to grade on Thursday morning).

The next major project — the assembly project from lecture 6 — is due in 13 days, on May 10th. Discussions are still due weekly, quizzes are open and repeatable, and the new course site (still unpublished) is on GitHub — check the pinned announcement.

What is a process?

A *process* is the bundle of context, code, and state that *is* an executing program: its memory, its open file descriptors, its registers, its position in the instruction stream, the works.

In the olden days, a process had *a* thread of execution — exactly one. Multi-user/multi-tasking systems gave each running program a single thread and the OS scheduled between them. The OS *still* schedules: each thread of execution gets a timeslice before being switched out. What's new (well, new-ish — this has been true for decades now) is that one process can host *multiple* threads of execution, and a real machine has multiple cores that can literally run threads in parallel.

If you want to peek at how Linux represents all this, the relevant header is `include/linux/sched.h`. The interesting fields on a `task_struct` for our purposes are `parent`, `thread_info`, and `pid`. Linux internally calls both processes and threads "tasks" — the distinction at the kernel level is mostly about what they share with their parent.

`fork`: the original concurrency primitive

`fork` spawns a new process that is a *clone* of the current one. Same code, same data, same open files, same next instruction. The implications are stranger than they sound:

- The new process picks up at the same line as the parent, because it has the same instruction pointer.
- There's no shared state between them. Modifying a variable in one does not change it in the other. (Modern kernels save space by having both processes point at the same physical memory pages until somebody writes — "copy-on-write" — but that's an implementation detail.)
- It's easy to write `fork`-style code that doesn't step on itself.
- It's *harder* to write `fork`-style code that coordinates and shares information, precisely because the state is separate.

`fork1.c` is the bare minimum:

```
printf("We haven't forked yet\n");
fork();
printf("We've now forked\n");
```

Run it and you'll see "We haven't forked yet" once and "We've now forked" *twice*. After `fork()` returns, there are two processes both sitting at the next line, both about to run that `printf`.

`fork2.c` makes the "no shared state" point concrete:

```
int num = 5;
fork();
num++;
printf("My number is: %d\n",num);
```

Each process has its own copy of `num`, so each one increments its own copy and prints `6`. Not `7`, not `6` and `7`: each process is bookkeeping its own state, full stop. (Aside: a polite program would call `wait()` in the parent so the child can finish before `main` returns. We're skipping that on purpose for now to keep the example short, but it's the kind of thing that bites you in shell pipelines.)

Process IDs and dispatching

If `fork` could only give you two identical clones running in lockstep, it wouldn't be very useful. The trick that makes it useful is that `fork` *returns* different things in the two processes:

- In the parent, `fork()` returns the child's PID (a positive integer).
- In the child, `fork()` returns `0`.
- On failure (e.g. out of process slots), it returns `-1` in the original process and there is no child.

`fork3.c` just prints the return value:

```
pid_t pid = fork();
printf("The value of pid is: %d\n",pid);
```

You'll see two lines: one with `0` (the child) and one with some big number (the parent's view of the child's PID).

`fork4.c` adds `getpid()`, which always returns "my own PID," so you can see who's who:

```
pid_t pid = fork();
printf("The value of pid is: %d\n",pid);
printf("My name is: %d\n",getpid());
```

The child's two lines will say "pid is 0" and "my name is `N`," where `N` is exactly the number the parent printed.

`fork5.c` is the canonical `fork` idiom: *branch on the return value* so parent and child do different things:

```
pid_t pid = fork();
if(pid != 0){
  printf("I'm the parent!\n");
```

```
}
else{
  printf("I'm the child!\n");
}
printf("The value of pid is: %d\n",pid);
printf("My name is: %d\n",getpid());
```

This is how every real `fork`-using program is shaped: the parent and the child take different branches and do different work. A shell calls `fork` to make a child, then has the child `exec` the command you typed while the parent waits. A web server forks one child per incoming connection. The pattern is always "one call, two return paths."

For contrast: threads

Threads are the other half of the picture. A thread of execution inside an existing process *shares* state with the other threads in that process: same heap, same globals, same file descriptors. That makes them lighter to spawn (no copy of the whole address space) and makes coordination through shared memory trivial — and it makes interfering with each other equally trivial. We'll get to that.

The headlines:

- Threads share state, processes don't.
- Spawning a thread is cheaper than spawning a process.
- Threads need explicit control flow: when you make one, you have to hand it a function to run, and the thread ends when that function returns.

That last point is the wedge for the next topic.

A word on function pointers

To launch a thread you have to give the threading library a *function* to run. C's way of saying "here, take this function" is a function pointer, which a lot of folks haven't seen before.

The point of function pointers is that they let you pass a function as a first-class argument. If you've used any modern language, you've probably used "higher-order functions" that take other functions — `map`, `filter`, `sort`-with-a-comparator, event handlers. C has had this since forever, just with crunchier syntax.

`funpoint1.c` is the warm-up:

```
int doubler(int n){
  return 2*n;
}

void maparr(int arr[], int s,
            int (*func)(int)){
  for(int i=0; i<5; i++){
    arr[i] = func(arr[i]);
  }
}

int main(){
```

```
  int arr[5] = {0,1,2,3,4};
  maparr(arr,5,doubler);
  for(int i=0; i<5; i++){
    printf("%d\n",arr[i]);
  }
}
```

The type `int (*func)(int)` reads as "a pointer to a function that takes an `int` and returns an `int`." The parentheses around `*func` matter — without them you'd be declaring a *function returning a pointer to int*, which is a totally different thing. Inside `maparr`, `func(arr[i])` calls through the pointer; you can also write `(*func)(arr[i])` if you want to be explicit, but the syntax sugar lets you drop the star.

The output is `0 2 4 6 8`, because `doubler` got applied to each element in place.

`pthread_create`: actually launching a thread

Now the threading API. The standard on Linux is POSIX threads (`pthreads`). The two calls you'll see today are `pthread_create`, which spawns a thread, and `pthread_join`, which waits for one to finish.

The signature of the thread function is fixed: `void* f(void* p)`. It takes a single `void*` argument and returns a `void*`. Why all the `void*`s? Because this is C's way of doing polymorphism: any pointer can be cast to `void*` and back, so the threading library can hand any function any kind of data without knowing what either of them is.

`thread1.c` is the simplest thing that works:

```
void* threadFun(void* p){
  printf("Hi from a thread!\n");
  return NULL;
}

int main(){
  pthread_t thread;
  pthread_create(&thread,NULL,threadFun,NULL);
  pthread_join(thread,NULL);
  return 0;
}
```

`pthread_create` takes:

1. A `pthread_t*` the library will fill in with the new thread's handle.
2. Optional attributes (`NULL` = defaults).
3. The function to run.
4. The single argument to pass it.

`pthread_join` blocks until the thread exits, and optionally captures its return value (we pass `NULL` because we don't care).

If you forget the join, `main` can return before the thread prints, and depending on timing the program ends without you ever seeing the message. Always join (or detach) your threads.

You'll also notice the `return NULL` at the end of `threadFun`. Returning a pointer when you have nothing to return is C's way of doing an "optional" value: a non-null pointer is your result, `NULL` means "nothing." Modern languages spell this `Maybe` or `Option`; in old-school C it's a pointer convention.

Passing data into a thread

`thread2.c` actually *uses* the argument:

```
void* threadFun(void* p){
  int* arg = (int*)p;
  (*arg)++;
  return NULL;
}

int main(){
  int counter = 0;
  pthread_t thread;
  pthread_create(&thread,NULL,threadFun,&counter);
  pthread_join(thread,NULL);
  printf("Counter is: %d\n",counter);
}
```

We pass `&counter` as the `void*` argument, the thread casts it back to `int*`, increments through it, and the parent sees `Counter is: 1` after the join. Crucially, this works because threads share memory: the thread's `*arg` and `main`'s `counter` are literally the same bytes. Try this with `fork` and you'd see no change in the parent — that's the whole difference.

The danger of shared state

Now the punchline of the lecture. `thread3.c` launches 100 threads, each of which increments `counter` 10,000 times:

```
void* threadFun(void* p){
  int* arg = (int*)p;
  int temp = *arg;
  for(int i=0; i<10000; i++){
    temp++;
  }
  *arg = temp;
  return NULL;
}

int main(){
  int counter = 0;
  pthread_t thread[100];
  for(int i=0; i<100; i++){
    pthread_create(&thread[i],NULL,threadFun,&counter);
  }
  for(int i=0; i<100; i++){
    pthread_join(thread[i],NULL);
  }
  printf("Counter is: %d\n",counter);
```

```
}
```

You'd hope, naively, for $100 \cdot 10{,}000 = 1{,}000{,}000$. You will not get 1,000,000. Run it a few times and you'll get different numbers, all far short.

The bug is the read-modify-write pattern in the body. Each thread:

1. reads `*arg` into a local `temp`,
2. does its 10,000 increments on `temp`,
3. writes `temp` back to `*arg`.

While thread A is doing its 10,000 increments, threads B, C, D, ... are also reading `*arg`, doing their increments, and writing back. Whoever writes last wins; everyone else's work is overwritten. This is a *race condition*, and it is the canonical failure mode of shared-state concurrency.

`thread4.c` is the same code with one tiny addition — a `usleep(100)` inside the inner loop, and only 100 iterations per thread instead of 10,000:

```
for(int i=0; i<100; i++){
  temp++;
  usleep(100);
}
```

The `usleep` forces the OS to interleave the threads more visibly — it's much harder for one thread to "win" the race by finishing its loop in a single timeslice. The result: `counter` ends up close to `100`, not `10,000`. Each of the 100 threads is essentially clobbering everyone else's writes.

The moral isn't "don't use threads." The moral is that *any time multiple threads touch the same memory, you need some form of synchronization* — mutexes, atomics, message passing, something. We covered the symptom today. The next chunk of the term is going to be about the cures.

8 Lecture 8 — Returning From Threads, Mutexes, and Critical Sections

Picking up where last lecture left us: we know how to spawn threads, we know how to pass them an argument, and we know that naïvely sharing state between them is a recipe for lost updates. Today's arc is the natural follow-up: how do you get a value *out* of a thread, what's the gotcha when you want to give a *group* of threads each their own argument, and what is this "mutex" thing that's supposed to fix the race condition? And then, just as importantly: when does fixing the race condition with a mutex *undo* the whole point of going concurrent in the first place?

Returning a value from a thread

We've passed data *into* threads via the `void*` argument. The flip side is the return value. Recall that the thread function's signature is `void* f(void* p)` — it returns a `void*`, and `pthread_join` can capture that pointer for you if you hand it a `void**`.

`threadRet.c` is the right way to do it:

```
void* worstIncrement(void* i){
  // 1. make space for the return value (malloc)
```

```
  // 2. put the value to be returned in it
  // 3. return the pointer
  int* p = malloc(sizeof(int));
  *p = *((int*)i)+1;
  return p;
}

int main(){
  pthread_t thread;
  int argVal = 10;
  void* retVal;
  pthread_create(&thread,NULL,worstIncrement,&argVal);
  pthread_join(thread,&retVal);
  printf("The returned value was: %d\n",*((int*)retVal));
  return 0;
}
```

The pattern is: `malloc` a fresh chunk of memory inside the thread, store the value there, and return the pointer. The caller joins, gets the pointer back via `retVal`, and can dereference it because that memory lives on the heap and outlives the thread that allocated it. (Yes, in real code you'd `free` it afterwards. We're keeping the example tight.)

Now the "why `malloc`" part. `threadRetBad.c` is the exact same idea but with the value living on the stack instead:

```
void* worstIncrement(void* i){
  int p; // lives in this stack frame, e.g. -4(%rbp)
  p = *((int*)i)+1;
  return &p; // returns a pointer into the about-to-die frame
}
```

The function returns a pointer to a local variable. The instant the function returns, that stack frame is conceptually gone — whatever the next function call does will reuse those bytes. By the time `main` dereferences `retVal` inside its `printf`, that address is pointing at junk. (Or, on a bad day, at memory the process isn't allowed to read, and you segfault.)

The whole reason we worked with the assembly view of stack frames earlier in the term is so this kind of bug stops being mysterious. "Returning the address of a stack local" isn't a vague hazard — it is concretely returning something like `-4(%rbp)` after `%rbp` has been restored to somebody else's frame.

A caveat on passing arguments to a group of threads

Last time we saw a single thread happily share an integer with `main`. Things get subtler when you spin up *many* threads and want each one to get its own distinct argument.

`threadArgs.c` is the right way:

```
void* sayHi(void* p){
  int id = *((int*)p);
  printf("Hi my name is: %d\n",id);
  return NULL;
}
```

```
int main(){
  pthread_t threads[20];
  int threadArgs[20];
  for(int i=0; i<20; i++){
    threadArgs[i] = i;
  }
  for(int i=0; i<20; i++){
    pthread_create(&threads[i],NULL,sayHi,threadArgs+i);
  }
  for(int i=0; i<20; i++){
    pthread_join(threads[i],NULL);
  }
  return 0;
}
```

Each thread gets the address of a *different* slot in the `threadArgs` array, and those slots have already been initialized to `0, 1, 2, ..., 19`. Each thread reads its own slot. Easy.

`threadArgsBad.c` is what you'd write if you weren't paying attention — pass each thread the address of the loop counter itself:

```
for(int i=0; i<20; i++){
  pthread_create(&threads[i],NULL,sayHi,&i);
}
```

Now every single thread is given the same pointer: `&i`. Because the loop is racing ahead while the threads are spinning up, each thread reads `*p` *whenever it happens to get scheduled*, which means by the time it actually runs, `i` may have already advanced. You'll see duplicate IDs, missing IDs, IDs of `20` (the post-loop value), all sorts of garbage. Worse, after `main` leaves the loop, `i` goes out of scope at some point, and now your threads are reading freed stack memory — the same hazard as the previous section, dressed up differently.

The fix and the bug both come from the same observation: in C, `&i` is the address of *the variable*, not a snapshot of its current value. If you want each thread to see a distinct value, you need a distinct *location* per thread. That's what the `threadArgs` array buys you.

Mutexes: the simplest fix for the race

Recall `thread3.c`/`thread4.c` from last lecture: 100 threads each doing a read-modify-write against a shared `counter`, and the final answer being absolutely nowhere near 100·100. `threadCounterBad.c` in this lecture's directory is the same idea, kept around for comparison:

```
void* threadFun(void* p){
  int* arg = (int*)p;
  int temp = *arg;
  for(int i=0; i<100; i++){
    temp++;
    usleep(100);
  }
  *arg = temp;
  return NULL;
}
```

Each thread reads `*arg`, increments a local copy 100 times (with `usleep` in there to make sure the OS interleaves aggressively), and writes the local copy back. They clobber each other.

The mechanical fix is a *mutex* — short for "mutual exclusion." A mutex is a little flag managed by the OS that exactly one thread at a time can "hold." If you try to lock a mutex that somebody else is already holding, your thread sleeps until the holder unlocks it.

`threadCounterMutex.c` bolts a mutex onto the bad version:

```
pthread_mutex_t m;

void* threadFun(void* p){
  int* arg = (int*)p;
  pthread_mutex_lock(&m);
  // first thread here runs through; everyone else waits
  // until somebody calls pthread_mutex_unlock
  int temp = *arg;
  for(int i=0; i<100; i++){
    temp++;
    //usleep(100);
  }
  *arg = temp;
  pthread_mutex_unlock(&m);
  return NULL;
}

int main(){
  pthread_mutex_init(&m, NULL);
  int counter = 0;
  pthread_t thread[100];
  for(int i=0; i<100; i++){
    pthread_create(&thread[i],NULL,threadFun,&counter);
  }
  for(int i=0; i<100; i++){
    pthread_join(thread[i],NULL);
  }
  printf("Counter is: %d\n",counter);
  pthread_mutex_destroy(&m);
  return 0;
}
```

The pattern to internalize:

- `pthread_mutex_init` once, before any threads see it.
- `pthread_mutex_lock` at the top of the protected region. This blocks if somebody else holds it.
- `pthread_mutex_unlock` at the bottom. *Every* path out of the protected region needs to unlock, or the next thread waits forever.
- `pthread_mutex_destroy` once, after all the threads have joined.

With the lock in place, the read-increment-write block becomes indivisible from any other thread's point of view. The final `counter` is now exactly $100 \cdot 100 = 10{,}000$, every single run.

`threadCounterMutex2.c` is the same code but with the `usleep(100)` re-enabled inside the loop. The point is to show that the answer is *still correct* — you just have to wait a lot longer for it. We'll come back to that in a minute.

Bundling the lock with the data

Putting the mutex in a global variable works, but in any nontrivial program you'd rather keep the lock and the data it protects together. `threadCounterStruct.c` does exactly that:

```
struct ThreadState {
  pthread_mutex_t mut;
  int counter;
};

void* threadFun(void* p){
  struct ThreadState* arg = (struct ThreadState*)p;
  pthread_mutex_lock(&arg->mut);
  int temp = arg->counter;
  for(int i=0; i<100; i++){
    temp++;
    usleep(100);
  }
  arg->counter = temp;
  pthread_mutex_unlock(&arg->mut);
  return NULL;
}

int main(){
  struct ThreadState s;
  s.counter = 0;
  pthread_mutex_init(&s.mut, NULL);

  pthread_t thread[100];
  for(int i=0; i<100; i++){
    pthread_create(&thread[i],NULL,threadFun,&s);
  }
  for(int i=0; i<100; i++){
    pthread_join(thread[i],NULL);
  }

  printf("Counter is: %d\n",s.counter);
  pthread_mutex_destroy(&s.mut);
  return 0;
}
```

Same answer, same correctness. The win is that the mutex and the counter it protects are now *one object*: if you pass the struct to another function, the lock goes with it. There's no globally visible `m` for somebody else's code to accidentally lock "the same" mutex against unrelated data. This is the shape you'll want for assignment 3.

The problem with overusing mutexes

Here's the trap. Wrapping more code in a mutex makes more bugs go away — so why not just wrap *everything*?

Look at `threadCounterMutex2.c` again. The lock is held across the entire 100-iteration loop, including the `usleep(100)` inside it. That means while thread A is sitting inside its loop (sleeping!), threads B, C, D, . . . , all 99 others, are blocked on `pthread_mutex_lock`, doing nothing. Each thread runs in strict sequence after the last one finishes.

If using mutexes too aggressively means each thread runs to completion before the next one starts, you've ended up with synchronous code with extra steps. The threads aren't buying you anything in terms of efficiency — if anything, things are *worse*, because you've got the cost of all those context switches and thread creations on top of a workload that is effectively a for loop.

That's pretty bad! And it leads directly to the next idea.

Critical sections

The *critical section* is the part of the code that runs between a lock and its corresponding unlock — the region only one thread can be inside at a time.

A big part of designing concurrent code well is figuring out what should and what *should not* be in the critical section. Things that have to be in there: any read-modify-write of shared state, any multi-step update where another thread reading the half-updated state would be wrong. Things that should *not* be in there: actual work. Number-crunching on local variables. I/O that doesn't touch shared state. Anything that can be done without seeing the shared data.

The mental rule of thumb: a critical section should be as small as possible while still being correct. You hold the lock just long enough to safely grab the next chunk of work or to commit the result of your chunk, and you do the chunk itself with the lock *not* held.

How this connects to assignment 3

Assignment 3 is going to ask you to process a very, very large array — think millions to hundreds of millions of elements — in parallel with a fixed pool of thread workers. If each thread does *all* of its work inside a critical section, you've just rebuilt `threadCounterMutex2.c` at scale, and it'll be ridiculously slow.

The trick is to figure out what actually needs to be guarded. The big hint: if you make sure each thread gets its *next chunk of work* assigned under a lock — so two threads never claim the same chunk — then the actual processing of that chunk can happen with no lock held at all. Locking the bookkeeping is fast; locking the work is not.

9 Lecture 9 — Semaphores, Dining Philosophers, Exceptions, and Signals

Last lecture wrapped up the "mutexes are the simple fix, but they serialize you if you're not careful" arc. Today is the natural next step: a sideways look at an alternative to locking (*transactional memory*), a more flexible cousin of the mutex (the *semaphore*), the canonical example of how locking can deadlock you (*dining philosophers*), and then a complete change of subject — how

the hardware and the kernel interrupt your process, and how your process can intercept those interruptions (*exceptions* and *signals*). Once we've got signals in hand, we'll use them as a crude but real form of inter-process communication.

Logistics

The assembly project is due in slightly under a week. *Please* don't start it on the last day — the bugs in this kind of project are mostly the slow, "oh, the calling convention got me" kind, and those eat hours. Work on it across this week if you haven't yet, so you can ask questions in office hours by Wednesday and have time to react to whatever I tell you.

Mutexes, redux

Quick reset on where we left off. A mutex is a way to guard a resource or a chunk of code: you *lock* before entering the critical section, the OS lets you in if nobody else is in, and otherwise puts you to sleep until the current holder unlocks. The crucial obligation is the matching *unlock* on every path out of the critical section. Forget the unlock and every thread that tries to enter that section after you blocks forever — the whole point of a thread yielding is that it expects to be woken up later, and "later" never arrives if nobody calls `unlock`.

That worst-case state has a name: *deadlock*. More generally, deadlock is any state of a concurrent system where some set of threads (or processes) is stuck waiting on each other and *no* thread can make progress. "Forgot to unlock" is the trivial way to get there, but you can construct subtler ones; we'll see one in detail today.

The same coda from last lecture still applies: the critical section should be as *small* as possible. Locking the bookkeeping is fast; locking the work itself rebuilds the synchronous version of the program with extra steps. That trade-off doesn't go away when we move to fancier synchronization primitives.

Transactional memory

Before we look at semaphores, it's worth pausing on a completely different paradigm for solving the same problem: *transactional memory*. The idea borrows directly from databases. Rather than "acquire a lock, do your update, release the lock," you say: "run this block of code as a transaction." The runtime tracks all the reads and writes you make inside the transaction, and at the end it either:

- *commits* your writes atomically, if no other transaction touched the same memory you read or wrote during your run; or
- *aborts* your work, throws away your writes as if they never happened, and re-runs your transaction from the top.

So instead of blocking, conflicting threads retry. There are two flavors out there:

- *Hardware transactional memory* (HTM): the CPU itself speculatively buffers writes and detects conflicts, e.g. Intel's TSX or IBM's Power8 implementation. It's fast when it works but has hard limits on transaction size.
- *Software transactional memory* (STM): the same idea in a library or language runtime. It looks more like the other user-space synchronization primitives we've been using (mutexes,

semaphores, etc.) and has no hard size limit, at the cost of more bookkeeping overhead.

The pitch for transactional memory is that you can't deadlock — there's no held resource that another thread is waiting on, because you don't hold anything until you commit. The catch is that you can still get stuck in something deadlock-*adjacent*: if two transactions keep conflicting and aborting each other in alternation, neither one ever makes progress. That's *livelock*, and it's its own can of worms.

We won't do TM hands-on in this class, but it's worth knowing the shape of the alternative. Locking is the most common strategy, not the only one, and it's not always the right one.

Semaphores

Semaphores are like mutexes except that, instead of having two states (locked / unlocked), they have a *counter* of resources. There are two operations on that counter:

- `sem_wait`: decrement the counter. If that would take it below zero, the calling thread blocks until somebody else increments it.
- `sem_post`: increment the counter. If anybody was blocked on `sem_wait`, one of them wakes up.

A semaphore initialized to `1` is *semantically* a mutex — one thread allowed past `sem_wait` at a time, everybody else queues up. That's the *binary semaphore* usage. The more interesting case is when you initialize the counter to something larger, which lets a semaphore guard *multiple equivalent resources* in a way a mutex can't. `semCounter.c` is the same counter-incrementing race we wrote with a mutex last lecture, ported to a semaphore:

```
sem_t s;

void* inc(void* arg){
  for(int i=0; i<100000; i++){
    sem_wait(&s);
    counter++;
    sem_post(&s);
  }
  return NULL;
}

int main(){
  sem_init(&s,0,1); // 0 = "shared between threads", 1 = initial count
  // ... spawn 1000 threads, each running inc, join them all ...
  sem_destroy(&s);
}
```

Final answer: $1000 \cdot 100000 = 100{,}000{,}000$, every run. Identical behavior to the mutex version — the API just looks different.

`semBouncer.c` is the canonical use of a counting semaphore: "bouncer at the door." 50 threads want to do work, but only some fixed number are allowed to be working concurrently:

```
sem_t s;

void* handler(void* arg){
```

```
  int id = *(int*)arg;
  sem_wait(&s); // grab a slot (blocks if all are taken)
  printf("Thread %d handling connection.\n", id);
  sleep(rand()%4 + 1);
  printf("Thread %d done.\n", id);
  sem_post(&s); // give the slot back
  return NULL;
}

int main(){
  sem_init(&s, 0, 6); // 6 slots
  // ... spawn 50 threads ...
}
```

Watch the output: at any moment exactly six threads are inside the "handling/done" block. The other 44 are queued on `sem_wait`. This is exactly the shape of a thread pool, a connection limiter, or any "at most N of these things at once" problem. Tweak the constant, observe the change in concurrency.

The other distinction worth remembering: a mutex is conceptually "owned" by the thread that locked it, and it's an error for a different thread to unlock it. A semaphore has no notion of ownership — one thread can `sem_post` a semaphore another thread is waiting on. That's a feature; it's how you use a semaphore for *signaling between threads*, as opposed to mutual exclusion.

The dining philosophers

The classic teaching example for deadlocks. Five philosophers sit at a round table. Between each pair of philosophers is a single fork. To eat, a philosopher needs both adjacent forks. They alternate between thinking (no forks needed) and eating (both forks needed).

Setup: $N = 5$ philosophers, N forks, modeled as an array of mutexes:

```
#define numPhilosophers 5
pthread_mutex_t forks[numPhilosophers];
```

The naive version. Each philosopher grabs left first, then right. `philo1.c`:

```
void* philosopher(void* num){
  int id = *(int*)num;
  while(1){
    sleep(1); // think
    pthread_mutex_lock(&forks[id]); // left
    sleep(1); // (so the deadlock has time to set up)
    pthread_mutex_lock(&forks[(id+1) % numPhilosophers]); // right
    // eat
    pthread_mutex_unlock(&forks[(id+1) % numPhilosophers]);
    pthread_mutex_unlock(&forks[id]);
  }
}
```

Run it. After the first round of "thinking," every philosopher grabs their left fork, and now every fork is held. Every philosopher is now blocked on their right fork (which their right neighbor is

holding). Nobody can ever proceed. Classic deadlock — and the shape of it is a *cycle* in the wait-for graph: philosopher 0 is waiting on a fork held by 1, who's waiting on 2, ..., who's waiting on 0. Break the cycle anywhere and you break the deadlock.

Fix #1: break the cycle. Have one philosopher reach in the opposite order. `philo2.c`:

```
if(id != numPhilosophers - 1){
  first = id;
  second = (id+1) % numPhilosophers;
} else {
  first = (id+1) % numPhilosophers;
  second = id;
}
// then lock first, lock second, eat, unlock second, unlock first
```

Now four philosophers reach left-then-right and one reaches right-then-left. The dependency graph no longer contains a cycle. Everybody eventually eats.

Fix #2: cap the table. Keep the naive ordering, but use a counting semaphore to make sure at most $N-1$ philosophers are even *trying* to grab forks at once. `philo3.c`:

```
sem_t seats; // initialized to numPhilosophers - 1 = 4

void* philosopher(void* num){
  int id = *(int*)num;
  while(1){
    sleep(1); // think
    sem_wait(&seats); // ask permission to come to the table
    pthread_mutex_lock(&forks[id]);
    pthread_mutex_lock(&forks[(id+1) % numPhilosophers]);
    // eat
    pthread_mutex_unlock(&forks[(id+1) % numPhilosophers]);
    pthread_mutex_unlock(&forks[id]);
    sem_post(&seats); // leave the table
  }
}
```

If only four philosophers are competing for forks, by pigeonhole at least one of them gets both forks. The fifth philosopher physically can't be in the "hold one fork and wait for the other" state — they're still outside the room, sleeping on `sem_wait(&seats)` — so the cycle never closes.

This second fix is structurally the same as the "`semBouncer`" example: a counting semaphore is being used to cap how many threads are inside a region of code. It just turns out that capping "how many philosophers are at the table" happens to be the right invariant to preserve.

Exceptions

Now we change subject. So far everything we've talked about has been voluntary scheduling — `fork`, `pthread_create`, `pthread_join`, etc. "Exceptions," in the systems sense (not the C++ sense), are the *involuntary* interruptions to your program: events that take control away from your code and hand it to the kernel.

Four classes of exceptions. A useful taxonomy:

- *Interrupts*: *asynchronous* — they come from outside the CPU. A network card finishing a packet receive, a keyboard press, a timer firing. They're not caused by your instruction stream; they just happen.
- *Faults*: *synchronous* and *potentially recoverable*. The CPU was trying to do something, couldn't, and raised an exception. A page fault is the classic example: the instruction tried to access an address whose page wasn't loaded; the kernel can fix it (load the page, return) and the instruction is retried.
- *Traps*: *synchronous* and *intentional*. The program ran an instruction *specifically* to hand control to the kernel. A system call (e.g. `syscall` on x86-64) is a trap: "please, kernel, do this for me."
- *Aborts*: *synchronous* and *unrecoverable*. The CPU is so confused (machine check, hardware error) that the only sensible thing left to do is tear the program down.

Most of what *your* code experiences is in the fault and trap buckets. Two faults you can trigger by hand:

`divByZero.c` — integer division by zero. The CPU itself raises an exception when `div` can't proceed; the kernel catches it and decides "this thread should die":

```
int x = 10, y = 0;
int z = x / y; // SIGFPE
```

`segfault.c` — the all-time favorite, dereferencing `NULL`:

```
int* p = NULL;
*p = 42; // SIGSEGV
```

In both cases, what's actually happening is:

1. Your instruction tries to do something the CPU can't complete.
2. The CPU raises an exception. Control jumps to the kernel.
3. The kernel inspects the situation, decides this isn't recoverable, and *signals* your process.

That last word is the bridge to the next topic.

Signals

A *signal* is the kernel's way of poking your process: "hey, something happened, deal with it." Each signal has a number and a default action. Some defaults are "terminate the process," some are "ignore," some are "stop the process until told to continue."

Sending signals from the terminal. The shell command `kill` doesn't necessarily kill anything — it sends signals. Some you'll see often:

- `SIGINT` (2) — the Ctrl-C signal. Default: terminate.
- `SIGTERM` (15) — a polite "please exit." Default: terminate. This is what `kill <pid>` sends if you don't specify a signal.

- `SIGKILL` (9) — the unblockable, uncatchable kill.
- `SIGSTOP` (19) — pause the process. Also unblockable.
- `SIGCONT` (18) — resume a stopped process.
- `SIGUSR1`, `SIGUSR2` — explicitly reserved for whatever you want to use them for.

`justSitHere.c` is a long-running program with no handlers installed. Run it in one terminal, find its PID, and try `kill -SIGINT <pid>`, `kill -SIGTERM <pid>`, etc. from another. Each one terminates it, because the defaults run.

Killing doesn't always make something dead. `cantStopMe.c` installs a handler for `SIGINT` and proceeds to *not* terminate when you Ctrl-C it:

```
void ouch(int sig){
  const char* msg = "\nOuch! That tickles. I'm not going anywhere.\n";
  write(1, msg, strlen(msg));
}

int main(){
  signal(SIGINT, ouch);
  while(1){
    printf("I'm Mr. %d and I cannot be stopped!\n", getpid());
    sleep(1);
  }
}
```

The only signals that you cannot intercept this way are `SIGKILL` and `SIGSTOP`. Those are guaranteed to work, which is why `kill -9` is the nuclear option in the system admin's toolkit.

Sending signals from inside a program. The `kill()` library call is just a thin wrapper around the syscall, taking the PID and the signal number. `sendSignal.c` is a hand-rolled version of the shell's `kill`:

```
int main(int argc, char** argv){
  int pid = atoi(argv[1]);
  int sig = atoi(argv[2]);
  kill(pid, sig);
}
```

`./sendSignal 12345 2` sends `SIGINT` to PID 12345. Pair it with `cantStopMe.c` from another terminal to drive the demo.

Receiving signals: the old way. The original API is `signal()`, and it's *very* simple:

```
signal(SIGUSR1, handler);
```

It's also full of historical sharp edges:

- On some old systems the handler was reset to the default after the first delivery, so you had to re-install it inside the handler itself.

- There's no way to say "while this handler is running, please block other signals." A second signal mid-handler can re-enter you.
- There's no way to ask for *automatic syscall restart*. A blocking syscall (`read`, `accept`, etc.) interrupted by a signal returns `EINTR` and you have to handle it yourself.

`sigOldWay.c` uses the simple form to count five `SIGUSR1` hits and then exit.

Receiving signals: the modern way. `sigaction()` is wordier but actually exposes the knobs you want:

```
struct sigaction sa;
sa.sa_handler = handler;
sigemptyset(&sa.sa_mask); // block no extra signals during handler
sa.sa_flags = SA_RESTART; // auto-restart interrupted syscalls
sigaction(SIGUSR1, &sa, NULL);
```

The three useful pieces:

- `sa_handler`: the function to run. Same role as the second arg to `signal()`.
- `sa_mask`: a set of signals to block *while this handler is running*, so a re-entrant signal can't trip you up.
- `sa_flags`: modifier flags. `SA_RESTART` is the biggie — it asks the kernel to resume any syscall that got interrupted by this signal.

`sigModernWay.c` is structurally identical to `sigOldWay.c` but built on `sigaction`. Use this one in real code.

Blocking signals. `sa_mask` blocks signals *while a handler is running*, but sometimes you want to block signals from regular code — for example, around a multi-step update to a global structure that the handler also reads. The API for that is `sigprocmask`:

```
sigset_t toBlock;
sigemptyset(&toBlock);
sigaddset(&toBlock, SIGINT);

sigprocmask(SIG_BLOCK, &toBlock, NULL); // start blocking SIGINT
// ... critical section: handler will not fire here ...
sigprocmask(SIG_UNBLOCK, &toBlock, NULL); // any pending one delivers now
```

The set type is `sigset_t`, built up with `sigemptyset`/`sigaddset`/`sigdelset`. The `how` argument is one of `SIG_BLOCK` (add to the blocked set), `SIG_UNBLOCK` (remove), or `SIG_SETMASK` (replace entirely).

`blockSignals.c` demos the behavior in three phases. Phase 1: no blocking, Ctrl-C delivers immediately and the handler prints. Phase 2: `SIG_BLOCK SIGINT`, mash Ctrl-C as much as you like — nothing happens. Phase 3: `SIG_UNBLOCK` and watch the backlogged signal deliver *exactly once*.

That last bit is the subtle thing worth emphasizing: standard (non-realtime) signals do not queue. The kernel maintains a single pending bit per signal per process. If `SIGINT` is blocked and arrives ten times, the first arrival sets the pending bit and the next nine are coalesced into the same bit.

When you unblock, the handler runs once. So you cannot use signal counts as a *message channel* where every arrival matters — if you need that, the realtime signals (`SIGRTMIN` through `SIGRTMAX`) do queue, but they're a different beast.

The other thing to know: `SIGKILL` and `SIGSTOP` can't be blocked. `sigprocmask` silently ignores attempts to add them to the blocked set, the same way you can't install a handler for them. The kernel reserves them precisely so there's always a way to take a process down or pause it.

Async-signal-safety. A signal handler runs at an arbitrary point in your program's execution, including in the middle of a libc call. If the handler then calls a libc function that isn't designed to be called that way, you can corrupt internal state. The list of *async-signal-safe* functions is short: `write`, `_exit`, `kill`, the simple `getpid`/`getppid`-style queries, and not much else. `printf` is *not* on that list. That's why all the handlers in these examples build a fixed string and call `write(1, msg, len)` instead of `printf`.

Using user-defined signals for IPC

`SIGUSR1` and `SIGUSR2` aren't claimed by the kernel for any purpose — they're yours. The most common use is signaling between a parent and a child after `fork`.

Re-visiting fork plus handlers. The mechanic to keep in mind is that after `fork()`, the child gets a *copy* of the parent's signal handler table. So if you install handlers *before* forking, both processes have the same handlers installed. `forkSignal.c` is a friendly ping/pong:

```c
volatile sig_atomic_t pinged = 0;

void onPing(int sig){ pinged = 1; /* write "pong!" */ }

int main(){
  // install handler with sigaction (modern way)
  sigaction(SIGUSR1, &sa, NULL);

  pid_t pid = fork();
  if(pid == 0){
    while(!pinged) pause(); // child waits for parent's ping
    kill(getppid(), SIGUSR1); // child replies
    return 0;
  }
  sleep(1); // give child time to settle
  kill(pid, SIGUSR1); // parent pings
  while(!pinged) pause(); // parent waits for child's reply
  wait(NULL);
}
```

A few things worth flagging:

- `volatile sig_atomic_t` is the type you reach for when a variable is shared between a handler and the rest of the code. `volatile` keeps the compiler from optimizing reads away ("you set this once and never write to it, so I'll cache it" would be a disaster here), and `sig_atomic_t` is the portable name for "a type whose loads and stores are guaranteed to be atomic with respect to signal delivery."

- `pause()` sleeps the calling process until *any* signal arrives. It's the right tool for "wait until something happens."

Two processes fighting with signals. `signalFight.c` is the same setup turned hostile. Two processes (parent and forked child) attack each other in alternation. The convention:

- `SIGUSR1` = "I just hit you, take damage."
- `SIGUSR2` = "I just died, you can stop fighting."

Each process keeps three pieces of state, all `volatile sig_atomic_t`: `hp`, `stillFighting`, `won`. The hit handler picks a random damage value, decrements `hp`, and if `hp` hits zero, sets the flags so the main loop will fall through and announce death.

```
void onHit(int sig){
  if(hp > 0){
    int dmg = rand() % 5 + 1;
    char buf[80];
    int n = sprintf(buf, "I, mr %d, took %d damage!\n", getpid(), dmg);
    write(STDOUT, buf, n);
    hp -= dmg;
    if(hp <= 0){ stillFighting = 0; won = 0; }
  }
}

void fightLoop(int enemy){
  while(stillFighting){
    // print hp
    kill(enemy, SIGUSR1);
    sleep(rand()%4 + 1);
  }
  kill(enemy, SIGUSR2);
  // if !won, print "I just died!"
}

int main(){
  // install handlers for SIGUSR1, SIGUSR2 with sigaction
  pid_t pid = fork();
  srand(time(0) ^ getpid()); // re-seed AFTER fork so the two roll differently
  if(pid == 0) fightLoop(getppid());
  else { fightLoop(pid); wait(NULL); }
}
```

A small but important detail: the seed gets set *after* the `fork()`, with `getpid()` mixed in. If we seeded before forking, both processes would have identical RNG state and would roll identical damage values. Same lesson as the "`&i` in a thread loop" bug from last lecture, in a different costume: inheriting state from the parent is great, except when you needed it to be different.

The output looks roughly like:

```
I, mr 6231, have 50 hp left
I, mr 6232, took 3 damage!
I, mr 6232, have 47 hp left
I, mr 6231, took 5 damage!
```

```
... etc., until one side hits 0 ...
I, mr 6232, just died!
```

This is, technically, IPC. Two separate processes coordinated their behavior without a shared variable, without a pipe, without shared memory — just `kill()` and signal numbers. It's a crude channel (one bit per signal type, no payload), but it does exist, and real programs do use it for coarse-grained coordination: "please reload your config" (`SIGHUP` is the convention), "please finish up and exit" (`SIGTERM`), and so on.

10 Lecture 10 — Homework Help and Project Time

No new material this day. We used the meeting as a lab session for assignment work, catching up on old homework, and answering whatever questions you had been sitting on. If you needed a checkpoint to ask "wait, what about *this*?", this was the day for it.

11 Lecture 11 — The Memory Hierarchy (CS:APP Ch. 6)

This lecture was an adaptation of Chapter 6 ("The Memory Hierarchy") of Bryant and O'Hallaron's *Computer Systems: A Programmer's Perspective*. There are no in-repo code samples to quote here — the material was the textbook's treatment of storage technologies, locality, the cache hierarchy, and the performance consequences of cache-friendly vs. cache-hostile access patterns. If you want to revisit it, work from CS:APP directly.

12 Lecture 12 — Virtual Memory (CS:APP Ch. 9)

This lecture was an adaptation of Chapter 9 ("Virtual Memory") of Bryant and O'Hallaron's *Computer Systems: A Programmer's Perspective*. Again, no class-specific code here — the source for this material is the textbook chapter, covering address translation, page tables, the TLB, demand paging, and how all of that interacts with `malloc`/`mmap`.

13 Lecture 13 — Files, File Descriptors, and the Two Layers of File IO

This was a full tour of how files actually work on Linux: the raw system-call layer (`open`, `close`, `read`, `write`), the higher-level C `FILE*` layer on top of it, and the various "everything is a file" consequences (reading directories, asking the kernel for metadata, and so on). We wrapped up by mashing all of it together into a tiny — and, true to the filename, terrible — line editor.

Logistics

A few reminders up front:

- If you're behind, you need to start catching up *right now*. The last call for old homework is Wednesday of "finals week," June 10th.
- Assignment 3 is due May 24th.
- The remaining discussions are PicoCTF (called Cylab now, apparently)

File descriptors

A file descriptor is just a non-negative integer. It indexes into some per-process bookkeeping the kernel maintains for you, but from your side of the line it's an `int`. Because the table is per-process, the kernel hands them out starting from the smallest unused slot, and the floor isn't zero: every program starts life with three descriptors already open.

- `0` — `stdin`
- `1` — `stdout`
- `2` — `stderr`

So the first file you `open()` typically comes back as `3`.

Command-line arguments, briefly

Before we got into `open`, we stopped to make sure everyone knew how `argv` actually works, because every example in this lecture takes a filename on the command line. `argv.c` is the one-screen demo:

```
int main(int argc, char* argv[]){
  printf("There are %d arguments and they are: \n", argc);
  for(int i=0; i<argc; i++){
    printf("Argument %d is %s\n", i, argv[i]);
  }
  return 0;
}
```

`argv[0]` is the program name as invoked, and `argv[1]` onward are whatever you typed after it on the shell.

Opening and closing the syscall way

The simplest possible program: take a filename as `argv[1]`, `open` it for reading, report the fd, close it.

```
#include <unistd.h>
#include <fcntl.h>
#include <stdlib.h>
#include <stdio.h>

int main(int argc, char* argv[]){
  if(argc < 2){ return 1; }

  int fd = open(argv[1], O_RDONLY);
  if(fd < 0){
    printf("Something went horribly wrong!\n");
  }
  else{
    printf("We opened the file at fd %d\n", fd);
    close(fd);
  }
  return 0;
}
```

`open` returns an `int`: a non-negative number on success (your shiny new fd) or `-1` on failure. The `O_RDONLY` flag is one of the three basic access modes.

To make the "smallest unused slot" rule visible, `openfile2.c` opens the same file in a loop and stashes each returned fd in an array. The descriptors come back as `3`, `4`, `5`, ...— one per call, monotonically increasing, because we never close in between.

```
int* fds = malloc(argc * sizeof(int));
for(int i=1; i < argc; i++){
  int fd = open(argv[1], O_RDONLY);
  printf("We opened the file at fd %d\n", fd);
  fds[i] = fd;
}
for(int i=1; i<argc; i++){
  if(fds[i] > 0){ close(fds[i]); }
}
free(fds);
```

Modes

The middle argument to `open()` is a bitwise-or of flags. The three you'll use constantly:

- `O_RDONLY` — read only
- `O_WRONLY` — write only
- `O_RDWR` — both

Add `O_CREAT` if you want the file created when it doesn't exist, and `O_TRUNC` if you want an existing file blanked to zero length before you start writing. When you pass `O_CREAT`, you also have to give a permission mode as the third argument — in octal, the same `0666`, `0644`, `0755` you'd use with `chmod`. `0666` means everyone gets read and write.

Reading: you almost always need a loop

`read()` doesn't promise to give you the whole file in one shot. It returns the number of bytes it actually read, which can be less than what you asked for, and it returns `0` at end of file. So the read pattern is: loop until the return is non-positive.

```
ssize_t bytesRead = 0;
char buff[1024];
while((bytesRead = read(fd, buff, sizeof(buff))) > 0){
  printf("We read in %lu bytes\n", bytesRead);
}
```

Writing: also a loop, for the same reason

`writefile1.c` takes a count and a filename and writes the numbers `0` through `count`, one per line. The syscall-side write looks like this:

```
int count = atoi(argv[1]);
int fd = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0666);

for(int i = 0; i <= count; i++){
```

```
  char buf[100];
  snprintf(buf, sizeof(buf), "%d\n", i);
  write(fd, buf, strlen(buf)); // strlen, not sizeof!
}
close(fd);
```

Live-coding catch: my first cut wrote `sizeof(buf)` instead of `strlen(buf)`, which would have written the entire 100-byte buffer (mostly garbage past the actual number) on every iteration. `sizeof` gives you the size of the array; `strlen` gives you the length of the C string inside it. Conflating these is a great way to corrupt files.

`writefile2.c` is the same program but with the proper write loop, because, just like `read`, `write` can do less than you asked:

```
char buf[100];
char* p = buf;
snprintf(buf, sizeof(buf), "%d\n", i);

int c = 0;
int remaining = strlen(buf);
while(((c = write(fd, p, remaining)) > 0) && remaining > 0){
  remaining = remaining - c;
  p = p + c;
}
```

The pattern: advance a pointer `p` past whatever got written, decrement `remaining`, keep going until either the call fails or there's nothing left. For small writes to a regular file you can almost always get away without the loop — but "almost always" is exactly the kind of bug that bites you on the day the disk is full or a signal arrives mid-call.

Everything is a file

The reason the same `read`/`write`/`open` API works for "a file on disk," "a pipe between two processes," and "a TTY" is that Linux exposes all of those as file descriptors. Two pseudo-files worth knowing about:

- `/dev/null` — a black hole. Writes succeed and are discarded; reads return EOF immediately. Use it when a program insists on writing somewhere and you don't care.
- `/dev/urandom` — an inexhaustible source of random bytes. Just `open` it and `read`; the kernel will keep handing you entropy as long as you ask.

Pipes (`|` in the shell) are the other classic example: the shell hands one process's `stdout` fd and another process's `stdin` fd to opposite ends of the same kernel buffer, and neither program has to know it's not talking to a real file.

The `FILE*` layer: same idea, friendlier

`open`/`read`/`write` are the raw system calls, and as we kept seeing, they have a lot of fiddly edges (write loops, `sizeof` vs. `strlen`, partial returns). The C standard library wraps them in a higher-level interface centered on `FILE*`, which gives you buffered I/O and a much friendlier set of helpers.

`openfileagain.c` reads a file line by line with `fgets`:

```
FILE* f = fopen("test.txt", "r");
char buf[100];
while(fgets(buf, sizeof(buf), f) > 0){
  printf("%s", buf);
}
```

And `weirdcopy.c` copies one file to another, again with the `FILE*` API:

```
FILE* inputFile = fopen(argv[1], "r+");
FILE* outputFile = fopen(argv[2], "w");

char line[1024];
while(fgets(line, 1024, inputFile) != NULL){
  fputs(line, outputFile);
}

fclose(inputFile);
fclose(outputFile);
```

The mode strings are the readable analogue of the syscall flags: `"r"` for read, `"w"` for write (truncates), `"a"` for append, and add a `"+"` for "and the other direction too." Under the hood `FILE*` is buffered: data you "write" sits in a userspace buffer until it's big enough to be worth handing to the kernel, which is why `fflush` and `fclose` exist and matter.

Reading directories

A directory is just another kind of file. The dedicated API for walking one is `opendir`/`readdir`/`closedir`, from `<dirent.h>`. `directory1.c` prints the regular files in a directory:

```
void listDir(const char* path) {
  DIR* dir = opendir(path);
  if (dir == NULL) {
    printf("Error opening directory: %s\n", strerror(errno));
    return;
  }
  struct dirent *entry;
  while ((entry = readdir(dir)) != NULL) {
    if(entry->d_type == DT_REG){
      printf("%s\n", entry->d_name);
    }
  }
  closedir(dir);
}
```

`readdir` hands back one `struct dirent*` per entry and `NULL` when it's done. The `d_type` field lets you filter by kind — `DT_REG` for regular files, `DT_DIR` for subdirectories, `DT_LNK` for symlinks, and so on.

Metadata: `stat`/`lstat`

If you want everything else the kernel knows about a file — size, permissions, owner, timestamps, what kind of file it actually is — you ask with `stat()` or `lstat()`. They fill in a `struct stat`

for you. The difference: `stat` follows symlinks to the thing they point at, `lstat` reports on the symlink itself.

`lstatTest.c` is essentially a tiny `ls -l`:

```
struct stat file_stat;
if(lstat(argv[1], &file_stat) == -1){
  printf("Oh something's wrong with that file.\n");
  return 1;
}

printf("File type: ");
switch (file_stat.st_mode & S_IFMT) {
  case S_IFREG: printf("regular file\n"); break;
  case S_IFDIR: printf("directory\n"); break;
  case S_IFLNK: printf("symbolic link\n"); break;
  case S_IFSOCK: printf("socket\n"); break;
  case S_IFBLK: printf("block device\n"); break;
  case S_IFCHR: printf("character device\n"); break;
  case S_IFIFO: printf("FIFO/pipe\n"); break;
  default: printf("unknown\n"); break;
}

printf("%c", file_stat.st_mode & S_IRUSR ? 'r' : '-');
printf("%c", file_stat.st_mode & S_IWUSR ? 'w' : '-');
printf("%c", file_stat.st_mode & S_IXUSR ? 'x' : '-');
/* ...group and other bits the same way... */

printf(" (%o)\n", file_stat.st_mode & 0777);
printf("Size of file in bytes: %ld\n", (long) file_stat.st_size);
printf("Access time: %s", ctime(&file_stat.st_atime));
printf("Modify time: %s", ctime(&file_stat.st_mtime));
printf("Change time: %s", ctime(&file_stat.st_ctime));
```

`st_mode` packs both the file type (the high bits, masked out by `S_IFMT`) and the permission bits (the low nine, masked out by `0777`) into a single integer. The `S_I*` macros are the named bit positions; `S_ISDIR(m)` and friends are convenience predicates that do the masking for you.

Putting it together: a tiny terrible line editor

`lineEditor.c` is the capstone. We open a file with the `FILE*` API, slurp every line into a `char**` array, and then loop on a menu: edit a line, delete a line, insert a line, or quit. On quit, we truncate the file back to zero and write the in-memory array out again.

```
void cleanUp(FILE* f, char** ls, int nL){
  ftruncate(fileno(f), 0);
  fseek(f, 0, SEEK_SET);
  for(int i=0; i < nL; i++){
    fputs(ls[i], f);
    free(ls[i]);
  }
  free(ls);
  fclose(f);
}
```

A pile of things worth pointing at in just this much code:

- `fileno(f)` extracts the underlying file descriptor from a `FILE*`, so you can mix the two APIs when you have to. `ftruncate` is a syscall and wants an fd, not a `FILE*`.
- `fseek(f, 0, SEEK_SET)` jumps the file position back to the start. The third argument is a whence: `SEEK_SET` (absolute), `SEEK_CUR` (relative to current), `SEEK_END` (relative to end). The syscall-layer equivalent is `lseek`.
- We truncate *and then* seek. Truncation changes the file's length, not the position pointer.

The slurp-and-edit loop is mostly bookkeeping:

```
char** lines = malloc(10000 * sizeof(char*));
int numLines = 0;

lines[0] = malloc(linesize);
while(fgets(lines[numLines], linesize, ourFile)){
  numLines++;
  lines[numLines] = malloc(linesize);
}
```

There is a mistake in my example where by trying to "always have a buffer ready for the next read" there is one extra unused `malloc` at the end, which we'd want to clean up in a less-terrible editor. It doesn't *really* matter but if you want to fix it you can.

Insert and delete are array-shift operations, like the ones you learned in your collegiate youth (e.g. last year). `insLine` shifts everything from the target index down by one and drops the new line in:

```
void insLine(int line, char** ls, int* nL){
  char* newLine = malloc(linesize * sizeof(char));
  printf("New text to insert at line %d:\n", line);
  while ((c = getchar()) != '\n' && c != EOF); // drain the input buffer
  fgets(newLine, linesize, stdin);
  for(int i = *nL; i > line; i--){
    ls[i] = ls[i-1];
  }
  ls[line] = newLine;
  (*nL)++;
}
```

The `while ((c = getchar()) != '\n' && c != EOF);` incantation is there because the previous `scanf("%d", ...)` that read the menu choice left the trailing newline sitting in the input buffer. Without draining it, `fgets` would happily "read" that newline and hand you back an empty line.

`delLine` is the mirror image — shift everything left, free the pointer that fell off the end — and `editLine` frees the old buffer at that index and replaces it with a freshly-read line.

The takeaway from the editor isn't really the editor: it's that every piece we'd built up over the lecture — `fopen`/`fclose`, `fgets`/`fputs`, `fseek`, mixing in the syscall layer with `fileno`/`ftruncate`, and the ever-present manual memory management — shows up in even a barely-functional real program.

14 Lecture 14 — "Everything is a File," IPC, and Shared Memory

This lecture pulled together a lot of threads: we leaned on the "everything is a file" philosophy to play with `/dev/urandom` and `/dev/null`, did a quick signals refresher (including handling `SIGSEGV`), and then worked our way through every flavor of inter-process communication Linux gives you — anonymous pipes, named pipes (FIFOs), Unix domain sockets, and shared memory via `mmap`, finally bolting a semaphore on top of the shared memory to make it race-free.

Everything is a file, continued

Linux's "everything is a file" bit isn't just a slogan; the same `open`/`read`/`write`/`close` API works for a regular file, a pipe, a socket, a terminal, a piece of hardware, or a chunk of the kernel pretending to be a file. Two of the most useful pseudo-files live in `/dev`.

`urandom.c` reads sixteen random bytes from `/dev/urandom`:

```
int fd = open("/dev/urandom", O_RDONLY);

unsigned char buf[16];
read(fd, buf, sizeof(buf));
close(fd);

printf("sixteen random bytes from the kernel: ");
for(int i = 0; i < 16; i++){
  printf("%02x", buf[i]);
}
printf("\n");
```

From your side of the line, this is exactly the same code you'd write to read a file off disk — it's just that the "file" on the other end is the kernel's CSPRNG, which will never run out of bytes and will never give you EOF. As an aside, if you're on a true Linux TTY (hit Ctrl-Alt-F3 on a real machine to get to one), you can literally draw graphics by `mmap`-ing the framebuffer device, because that too is just a file.

`devnull.c` is the mirror image: `/dev/null` is the black hole. Writes are accepted and discarded; reads return EOF immediately.

```
// shout into the void as much as we want
int sink = open("/dev/null", O_WRONLY);
for(int i = 0; i < 1000; i++){
  char msg[64];
  int n = snprintf(msg, sizeof(msg), "screaming: %d\n", i);
  write(sink, msg, n);
}
close(sink);

// reading is even shorter: 0 bytes, immediately
int empty = open("/dev/null", O_RDONLY);
char buf[100];
ssize_t got = read(empty, buf, sizeof(buf));
printf("read from /dev/null returned %ld bytes\n", (long)got);
close(empty);
```

`/dev/null` is the thing you point a program at when it insists on writing somewhere and you don't care — the shell trick `cmd > /dev/null 2>&1` is exactly this idea.

IPC, in general

Inter-process communication is the umbrella term for "how do two processes talk to each other." By default, they can't: each process gets its own virtual address space, so the parent's pointers aren't even meaningful to the child after `fork`. To share anything, you have to go through the kernel. Linux gives you a buffet of ways to do it, and we walked through most of them:

- Signals (last lecture, briefly revisited)
- Anonymous pipes (parent/child only)
- Named pipes / FIFOs (any two processes, by filename)
- Unix domain sockets (any two processes, full-duplex)
- Shared memory via `mmap` (with a semaphore on top)

Signals, a quick refresher

A signal is the kernel poking your process: "something happened, deal with it." Some signals are mandatory and fatal by default (`SIGSEGV`, `SIGFPE`); some you can install a handler for and override the default. `SIGUSR1` and `SIGUSR2` have no built-in meaning at all, which is exactly what makes them useful for cross-process notification you control.

`signalDuel.c` is our toy example: a parent and child fork each other, then duel by sending `SIGUSR1` "hits." Each process tracks 10 hp in a `volatile sig_atomic_t`, and the loser sends `SIGUSR2` to the winner to end the game.

```c
volatile sig_atomic_t hp = 10;
volatile sig_atomic_t fighting = 1;

void onHit(int sig){
  hp--;
  if(hp <= 0) fighting = 0;
}

void endGame(int sig){
  fighting = 0;
  char msg[] = "Oh, I guess I won?\n";
  write(1,msg,sizeof(msg));
}

int main(){
  // install handlers BEFORE fork so both processes inherit them
  signal(SIGUSR1, onHit);
  signal(SIGUSR2, endGame);

  pid_t pid = fork();
  pid_t enemy = (pid == 0) ? getppid() : pid;
  srand(time(NULL) ^ pid); // xor so the two seeds differ

  while(fighting){
```

```
    kill(enemy, SIGUSR1);
    int left = rand()%3+1;
    while((left = sleep(left)) > 0); // sleep returns time *remaining*
    if(fighting){
      printf("pid %d: hp = %d\n", getpid(), hp);
    }
  }

  if(hp <= 0){
    printf("pid %d: I died!\n", getpid());
    kill(enemy, SIGUSR2);
  }

  if(pid > 0) wait(NULL);
  return 0;
}
```

A few things worth pointing at:

- `volatile sig_atomic_t` is the only kind of variable you're allowed to touch from inside a signal handler safely. The `volatile` stops the compiler from caching it in a register; the `sig_atomic_t` type promises the read/write is atomic with respect to signal delivery.
- Inside a handler you should only call "async-signal-safe" functions. `write` is on that list; `printf` is not — it can deadlock if the signal interrupted a previous `printf` mid-buffer-flush. That's why `endGame` uses `write(1, msg, sizeof(msg))` instead of `printf`.
- `sleep` returns the number of seconds *left* if it gets interrupted by a signal. The `while((left = sleep(left)) > 0)` idiom restarts the nap with whatever's left over. If you just write `sleep(left)` once, every signal cuts your sleep short — and in a duel where you're being hit constantly, that's basically always.

Catching the uncatchable-looking

`segfault.c` is the un-elaborate version: dereference `NULL`, watch the program die.

```
int* p = NULL;
printf("about to dereference NULL\n");
*p = 42;
printf("never gets here\n");
```

The kernel sees the page fault on an unmapped page, can't recover, and signals you with `SIGSEGV`. The default handler prints "Segmentation fault" and kills you.

You can install your own handler for `SIGSEGV` — `handleSegfault.c` does — but it's almost always a bad idea, because by the time you get there your address space is in an unknown state. The most you should do is print a message and exit:

```
void segfun(int sig){
  const char msg[] = "This segmented too close to the sun\n";
  write(1, msg, sizeof(msg)-1);
  _exit(1);
}
```

```c
int main(){
  int* boop = NULL;
  signal(SIGSEGV, segfun);
  *boop = 50;
  return 0;
}
```

`_exit` (with the underscore) skips the libc cleanup that regular `exit` does — no flushing buffers, no running `atexit` handlers — which is the right move when you don't trust the state of your own process.

Anonymous pipes

A pipe is a one-way, in-kernel byte buffer with a read end and a write end. `pipe(fds)` fills in a two-element `int` array: `fds[0]` is the read end, `fds[1]` is the write end. By itself this is useless (one process talking to itself), but if you `fork` *after* creating the pipe, both ends get inherited by both processes, and now they can talk.

`anonPipe.c` is the minimal example: parent writes one string, child reads it.

```c
int fds[2];
pipe(fds);

pid_t pid = fork();
if(pid == 0){
  // child reads
  close(fds[1]); // child doesn't write, so close the write end
  char buf[128];
  ssize_t n = read(fds[0], buf, sizeof(buf) - 1);
  buf[n] = 0; // read doesn't null-terminate
  printf("child read: %s", buf);
  close(fds[0]);
} else {
  // parent writes
  close(fds[0]);
  const char* msg = "hello from the parent, down a pipe\n";
  write(fds[1], msg, strlen(msg));
  close(fds[1]); // closing the write end is what gives the child its EOF
  wait(NULL);
}
```

Two things to internalize:

- Close the end you aren't using. If the parent leaves the read end open, then even after *it* closes the write end the kernel still sees an open write end (its own), and the child will block on `read` forever waiting for an EOF that never comes.
- `read` doesn't put a `\0` at the end of the buffer for you. You have to do it yourself based on the return value, which is how many bytes actually got read.

`anonpipescanf.c` is the same thing but the parent reads its message from `scanf` first, so you can type something at the keyboard and watch it appear on the other side of a pipe. The only structural change is the parent's write half.

```
close(fds[0]);
char msg[100];
printf("What am I saying to the child?: ");
scanf("%s", msg);
write(fds[1], msg, strlen(msg));
close(fds[1]);
wait(NULL);
```

`anonpipeloop.c` extends this to a conversation: the parent loops on `scanf`, sends each word down the pipe, and the child loops on `read` echoing whatever shows up. Type `quit` to stop.

```
if(pid == 0){
  close(fds[1]);
  char buf[128];
  ssize_t n;
  while((n = read(fds[0], buf, sizeof(buf) - 1)) > 0){
    buf[n] = 0;
    printf("\nchild echo: %s\n", buf);
  }
  close(fds[0]);
} else {
  close(fds[0]);
  char msg[100];
  while(1){
    printf("Mr. %d: What am I saying to the child? (type quit to stop): ", getpid());
    scanf("%s", msg);
    if(strcmp(msg, "quit") == 0) break;
    write(fds[1], msg, strlen(msg));
  }
  close(fds[1]);
  wait(NULL);
}
```

This works, but the user experience is awful: the parent's prompt and the child's echo race each other to `stdout`, so the output is a jumble. The fix is the *two-pipe trick*, in `anonPipeLoopFix.c`: one pipe parent→child for the message, a second pipe child→parent for an acknowledgement. The parent waits for the ack before printing the next prompt, and the output stays orderly.

```
int p2c[2]; // parent -> child
int c2p[2]; // child -> parent
int ready = 1;
pipe(p2c);
pipe(c2p);

pid_t pid = fork();
if(pid == 0){
  close(p2c[1]); close(c2p[0]);
  char buf[128];
  ssize_t n;
  while((n = read(p2c[0], buf, sizeof(buf) - 1)) > 0){
    buf[n] = 0;
    printf("child echo: %s\n", buf);
    write(c2p[1], &ready, sizeof(int)); // tell parent: go ahead
```

```
  }
} else {
  close(p2c[0]); close(c2p[1]);
  char msg[100];
  while(1){
    printf("Mr. %d: What am I saying to the child? (type quit to stop): ", getpid());
    scanf("%s", msg);
    if(strcmp(msg, "quit") == 0) break;
    write(p2c[1], msg, strlen(msg));
    read(c2p[0], &ready, sizeof(int)); // wait for ack
  }
  close(p2c[1]);
  wait(NULL);
}
```

You can think of `c2p` here as a one-bit semaphore built out of a pipe — the parent blocks on `read` until the child writes, and the actual contents of `ready` don't matter.

Named pipes (FIFOs)

Anonymous pipes only work between related processes, because the file descriptors only show up across `fork`. *Named* pipes solve that by giving the pipe a name in the filesystem — something you can `open` from any process that knows the path.

You can even make one from the shell:

```
mkfifo /tmp/myFifo
ls -l /tmp/myFifo # type starts with 'p' for pipe
```

`namedPipeWriter.c` creates the FIFO (if it doesn't exist) and writes a few messages to it:

```
mkfifo("/tmp/myFifo", 0666); // harmless if it already exists

printf("waiting for a reader on /tmp/myFifo...\n");
int fd = open("/tmp/myFifo", O_WRONLY);
printf("reader connected, sending messages\n");

for(int i = 0; i < 5; i++){
  char msg[64];
  int n = snprintf(msg, sizeof(msg), "message %d down the pipe\n", i);
  write(fd, msg, n);
  sleep(1);
}
close(fd);
```

`namedPipeReader.c` is the matching reader:

```
printf("waiting for a writer on /tmp/myFifo...\n");
int fd = open("/tmp/myFifo", O_RDONLY);
printf("writer connected, reading messages\n");

char buf[128];
ssize_t n;
while((n = read(fd, buf, sizeof(buf) - 1)) > 0){
```

```
  buf[n] = 0;
  printf("got: %s", buf);
}
close(fd);
```

Run the reader in one terminal and the writer in another and watch them connect. The classic gotcha: `open` on a FIFO *blocks* until the other side shows up. That's a feature, not a bug — it's how the two ends rendezvous — but it surprises people the first time.

Once they're connected, this looks and behaves exactly like a regular file: `ls -l /tmp/myFifo` shows the entry, it has permissions, and it shows up in `/proc/<pid>/fd` on the opened end. That's the "everything is a file" philosophy again.

Unix domain sockets

Sockets are the general API for talking over a channel, with a much richer set of operations than pipes (full duplex, message boundaries on `SOCK_DGRAM`, peer credentials, ancillary data, etc.). The Internet ones, `AF_INET`, are how you talk to another machine. The local ones, `AF_UNIX`, give you the same API but staying entirely inside the kernel, with a path in the filesystem as the address.

The server (`domainServer.c`) does the four-call dance: create, bind, listen, accept.

```
unlink("/tmp/mySock"); // clear any stale socket from a previous run

int s = socket(AF_UNIX, SOCK_STREAM, 0);

struct sockaddr_un addr;
memset(&addr, 0, sizeof(addr));
addr.sun_family = AF_UNIX;
strcpy(addr.sun_path, "/tmp/mySock");

bind(s, (struct sockaddr*)&addr, sizeof(addr));
listen(s, 1); // backlog: how many pending clients to queue

printf("waiting for a client on /tmp/mySock...\n");
int conn = accept(s, NULL, NULL); // returns a DIFFERENT fd
printf("client connected\n");

char buf[256];
ssize_t n;
while((n = read(conn, buf, sizeof(buf) - 1)) > 0){
  buf[n] = 0;
  printf("client says: %s", buf);
}

close(conn);
close(s);
unlink("/tmp/mySock"); // clean up after yourself
```

The key thing students miss: `accept` hands you a *new* file descriptor for the actual conversation. The original socket `s` is the listening one; you keep it open if you want to accept more clients later. The new `conn` is what reads and writes happen on. This same pattern — `socket`/`bind`/`listen`/`accept` — is what you'll use for Internet sockets too.

The client (`domainClient.c`) is shorter: create, connect, talk.

```
int s = socket(AF_UNIX, SOCK_STREAM, 0);

struct sockaddr_un addr;
memset(&addr, 0, sizeof(addr)); // zero the whole struct first
addr.sun_family = AF_UNIX;
strcpy(addr.sun_path, "/tmp/mySock");

connect(s, (struct sockaddr*)&addr, sizeof(addr));

for(int i = 0; i < 5; i++){
  char msg[64];
  int n = snprintf(msg, sizeof(msg), "ping %d from the client\n", i);
  write(s, msg, n);
  sleep(1);
}
close(s);
```

And, again, "everything is a file": while the server is running, `ls /proc/<server_pid>/fd` shows the listening socket and the accepted connection as fds, sitting next to `stdin`, `stdout`, `stderr`.

Shared memory with `mmap`

`mmap` maps a region of memory into your address space. What's on the other end depends on how you call it:

- Pass a real file descriptor: the mapping is backed by that file, and reads/writes to the memory turn into reads/writes to the file (when the kernel decides to flush, or when you call `msync`).
- Pass `-1` with `MAP_ANONYMOUS`: there's no file; the mapping is just zeroed pages. This is, in spirit, "a big and weirder `malloc`."
- Pass `MAP_SHARED` and the mapping is visible to other processes that map the same backing object. `MAP_PRIVATE` keeps changes copy-on-write and local.

`mmap` on a file, for reading

`mmapFile.c` maps a whole file read-only and then dumps it:

```
int fd = open(argv[1], O_RDONLY);

struct stat st;
fstat(fd, &st);

char* contents = mmap(NULL, st.st_size,
                      PROT_READ, MAP_PRIVATE,
                      fd, 0);

printf("file is %ld bytes\n", (long)st.st_size);
printf("first byte: '%c'\n", contents[0]);
printf("last byte: '%c'\n", contents[st.st_size - 1]);
write(1, contents, st.st_size);
```

```
munmap(contents, st.st_size);
close(fd);
```

We need `fstat` to ask the kernel how big the file is, so we know how much to map. `PROT_READ` says the memory is read-only; `MAP_PRIVATE` says any writes (if we somehow tried) would stay local to this process. The first argument is `NULL` for "let the kernel pick the address."

`mmap` on a file, for writing

To actually modify the file through the mapping, two things have to change: open the file `O_RDWR`, and map it with `PROT_READ | PROT_WRITE | MAP_SHARED`. `screamFile.c` does exactly that, then replaces every non-newline byte with `'A'`:

```
int fd = open(argv[1], O_RDWR);

struct stat st;
fstat(fd, &st);

char* contents = mmap(NULL, st.st_size,
                      PROT_READ | PROT_WRITE, MAP_SHARED,
                      fd, 0);

// turn the file into nothing but screaming
for(int i = 0; i < st.st_size; i++){
  if(contents[i] != '\n'){
    contents[i] = 'A';
  }
}

write(1, contents, st.st_size);
munmap(contents, st.st_size);
close(fd);
```

After `munmap`, the file on disk has been rewritten — the kernel propagates writes to the backing file because we said `MAP_SHARED`. With `MAP_PRIVATE` the kernel would have given us a copy-on-write page instead and the file would have been untouched.

Run it on `toscream.txt` (which starts as a few lines of mixed text) and the file becomes:

```
AAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAA
AAAAAAAAAAAAAA
```

`mmap` to share memory between processes

If we drop the file backing and use `MAP_ANONYMOUS | MAP_SHARED`, we get a chunk of zeroed memory that the kernel will share between us and any process we `fork` afterwards. That's how `mmapShared.c` lets a parent and child share a counter:

```
int* counter = mmap(NULL, sizeof(int),
                    PROT_READ | PROT_WRITE,
                    MAP_SHARED | MAP_ANONYMOUS, -1, 0);
*counter = 0;
```

```
pid_t pid = fork();
if(pid == 0){
  for(int i = 0; i < 100000; i++){
    (*counter)++;
  }
  return 0;
} else {
  for(int i = 0; i < 100000; i++){
    (*counter)++;
  }
  wait(NULL);
  printf("final counter (expected 200000): %d\n", *counter);
}
```

And... we get the same horror story we got with threads several lectures ago. The counter is wildly under 200000 most runs, because `(*counter)++` is a load-modify-store, the two processes interleave, and updates get lost. Sharing memory between processes brings back every concurrency hazard you thought you'd escaped by leaving threads behind.

Fixing it with a semaphore

The cure is the same as it was for threads, except mutexes are process-local — they live in the locking process's address space and don't synchronize across a `fork`. *Semaphores* can be marked process-shared (`pshared = 1`) and placed in shared memory, and then they work across processes.

`mmapSem.c` puts the counter and a semaphore together in one shared struct:

```
struct shared {
  sem_t lock;
  int counter;
};

int main(){
  struct shared* s = mmap(NULL, sizeof(struct shared),
                          PROT_READ | PROT_WRITE,
                          MAP_SHARED | MAP_ANONYMOUS, -1, 0);

  sem_init(&s->lock, 1, 1); // pshared = 1, initial value = 1
  s->counter = 0;

  pid_t pid = fork();
  if(pid == 0){
    for(int i = 0; i < 100000; i++){
      sem_wait(&s->lock);
      s->counter++;
      sem_post(&s->lock);
    }
    return 0;
  } else {
    for(int i = 0; i < 100000; i++){
      sem_wait(&s->lock);
      s->counter++;
```

```
      sem_post(&s->lock);
    }
    wait(NULL);
    printf("final counter (should be 200000): %d\n", s->counter);
  }

  sem_destroy(&s->lock);
  munmap(s, sizeof(struct shared));
  return 0;
}
```

The two key bits:

- Both the `counter` and the `sem_t` live inside the same `mmap`'d struct, so both processes see the same semaphore as well as the same counter. Putting the `sem_t` on the parent's stack would defeat the purpose entirely — the child would have its own copy after `fork` and they wouldn't synchronize with anything.
- The second argument to `sem_init` is the `pshared` flag. Passing `1` (and putting the semaphore in shared memory) makes it process-shared; passing `0` keeps it thread-only.

With the semaphore in place the final counter prints `200000` every run, and we've closed the loop: shared memory plus a process-shared semaphore is essentially the threads playbook, just with processes instead of threads and with the kernel arbitrating the shared region instead of the runtime.

15 Lecture 15 — Internet Sockets: Echo Servers, Web Servers, and Handling Many Clients

This is the last lecture of new content for the class, and it's the one where all the IPC stuff from last time grows up and learns to talk over a network. Unix domain sockets gave us the `socket`/`bind`/`listen`/`accept` dance with a path in the filesystem as the address; *Internet* sockets are the same four calls with an IP address and a port instead. We built up from a one-client echo server to a forking one, a threading one, a tiny web server you can hit from a real browser, and finally the classic `select`/`poll` way of juggling many clients in a single thread. The definitive reference for all of this is Beej's Guide to Network Programming (`https://beej.us/guide/bgnet/`), and there's a `socketsGuide.org` in this repo too.

Network byte order vs. host byte order

Remember endianness from way back at the start of the class? It's about how the bytes of a multi-byte number are laid out in memory. Take `0x123456789ABCDEFF` (eight bytes):

- **Big-endian**: the lowest address holds the most-significant byte, `0x12`.
- **Little-endian**: the lowest address holds the least-significant byte, `0xFF`.

The catch is that the network has its own convention — "network byte order" is big-endian — but not every machine agrees with it. x86-64, the thing you're almost certainly running on, is *always* little-endian. So any multi-byte value you put into a packet header (most importantly the port number) has to be converted first. That's what these four functions are for:

- `htons`/`htonl` — **h**ost **to** **n**etwork, **s**hort or **l**ong
- `ntohs`/`ntohl` — the reverse

You'll see `htons(port)` on every `sin_port` assignment below and `htonl(INADDR_ANY)` on every `sin_addr`. On a big-endian machine these would be no-ops; on yours they actually swap bytes. Writing them anyway makes the code portable.

TCP vs. UDP (we're only doing TCP)

Everything in this lecture uses TCP, which is the `SOCK_STREAM` you'll see passed to `socket`. The short version of the difference:

- **TCP** guarantees every byte arrives, exactly once, in order. It does this with sequence numbers and acknowledgements (ACKs) under the hood, retransmitting anything that gets lost. You get a reliable byte stream.
- **UDP** doesn't bother with any of that. Packets can be dropped, duplicated, or reordered, and it's on you to cope. The upside is lower overhead, which is why it's used for things like video calls where a late packet is worse than a lost one.

Because TCP is a byte *stream*, there are no message boundaries — a single `recv` might hand you half a message or three messages glued together. For our echo servers that doesn't matter (we send back whatever bytes we got), but it's the first thing that bites people writing a real protocol.

A one-client echo server

`echoServerOnce.c` is the smallest thing that does the full dance: create a socket, bind it to a port, listen, accept exactly one client, echo until they leave, then exit.

```
int s = socket(AF_INET, SOCK_STREAM, 0);

// SO_REUSEADDR lets us re-bind to the same port immediately after we
// exit; without it the kernel holds the port in TIME_WAIT and bind()
// fails with "Address already in use" for a while
int yes = 1;
setsockopt(s, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof(yes));

struct sockaddr_in addr;
memset(&addr, 0, sizeof(addr));
addr.sin_family = AF_INET;
addr.sin_addr.s_addr = htonl(INADDR_ANY); // listen on every interface
addr.sin_port = htons(port); // network byte order!

bind(s, (struct sockaddr*)&addr, sizeof(addr));
listen(s, 1);

int conn = accept(s, NULL, NULL); // a brand-new fd for this conversation

char buf[1024];
ssize_t n;
while((n = recv(conn, buf, sizeof(buf), 0)) > 0){
  send(conn, buf, n, 0);
}
```

```
close(conn);
close(s);
```

A few things to anchor on:

- Those casts to `(struct sockaddr*)` look bizarre. They're there because these interfaces are ancient: there's one generic `struct sockaddr` that the calls take, and a pile of more-specific structs (`sockaddr_in` for IPv4, `sockaddr_un` for Unix domain) that you cast *down* to it. It's a hand-rolled kind of subtyping from before C had anything better.
- `accept` returns a *different* fd from the one you're listening on — exactly like the Unix domain socket case. `s` is the doorbell; `conn` is the actual conversation.
- `recv` and `send` are just `read` and `write` with an extra flags argument for socket-specific options. We pass `0` and they behave identically.
- `recv` returning `0` means the client closed their end — that's our EOF, and it's how the loop terminates.

You can talk to it with `nc localhost 5000` (netcat is a generic "connect a socket to my terminal" tool), or with our own `echoClient.c`, which is the mirror image: create, connect, then loop sending lines and printing what comes back.

```
int s = socket(AF_INET, SOCK_STREAM, 0);

struct sockaddr_in addr;
memset(&addr, 0, sizeof(addr));
addr.sin_family = AF_INET;
addr.sin_port = htons(atoi(argv[1]));
// inet_pton parses a dotted-quad string ("127.0.0.1") into the
// binary form the kernel wants; it returns 1 on success
inet_pton(AF_INET, "127.0.0.1", &addr.sin_addr);

connect(s, (struct sockaddr*)&addr, sizeof(addr));

char line[1024];
while(fgets(line, sizeof(line), stdin)){
  send(s, line, strlen(line), 0);
  char buf[1024];
  ssize_t n = recv(s, buf, sizeof(buf) - 1, 0);
  if(n <= 0) break;
  buf[n] = 0;
  printf("server said: %s", buf);
}
close(s);
```

The client has no `bind`/`listen`/`accept` — it just `connect`s. `inet_pton` ("presentation to network") is the function that turns the human string `"127.0.0.1"` into the packed 32-bit address the kernel stores in `sin_addr`.

An acceptance loop: serving clients back-to-back

`echoServerOnce.c` handles exactly one client and quits. The fix in `echoServerLoop.c` is almost embarrassingly small: wrap the `accept` in a `while(1)` and *never* close the listening socket. When one client hangs up, we loop back around and accept the next.

```
while(1){
  struct sockaddr_in caddr;
  socklen_t clen = sizeof(caddr);
  int conn = accept(s, (struct sockaddr*)&caddr, &clen);
  if(conn < 0){ perror("accept"); continue; }

  // who connected? accept filled in caddr for us
  char ip[INET_ADDRSTRLEN];
  inet_ntop(AF_INET, &caddr.sin_addr, ip, sizeof(ip));
  printf("connection from %s:%d\n", ip, ntohs(caddr.sin_port));

  char buf[1024];
  ssize_t n;
  while((n = recv(conn, buf, sizeof(buf), 0)) > 0){
    send(conn, buf, n, 0);
  }
  close(conn); // close the CLIENT, not the listener
}
```

This time we pass `accept` a `caddr` to fill in, so we can see who connected. `inet_ntop` is the inverse of `inet_pton` — it turns the binary address back into a printable string, and `ntohs` flips the port back to host order for the `printf`.

The important limitation: this is still completely *serial*. A second client that connects while we're busy echoing for the first one doesn't get rejected — it sits in the listen backlog (that's the `2` in `listen(s, 2)`) waiting its turn. We don't call `accept` on it until the first client leaves. To handle two clients at the same time we need to stop blocking the whole program on one conversation.

One process per client: fork

The first way to get concurrency is the one we already know: `fork` a child for each connection and let the child run the echo loop while the parent goes straight back to `accept`. `echoServerFork.c`:

```
// "when a child exits, don't make me reap it" -- avoids zombies
// without us having to wait() on every child
signal(SIGCHLD, SIG_IGN);

// ... socket / bind / listen as before ...

while(1){
  int conn = accept(s, (struct sockaddr*)&caddr, &clen);
  if(conn < 0){ perror("accept"); continue; }

  pid_t pid = fork();
  if(pid == 0){
    close(s); // child doesn't need the listener
    echoLoop(conn);
    close(conn);
```

```
    _exit(0);
  } else {
    close(conn); // parent doesn't need this client's fd
  }
}
```

The weird trick the agenda warned about is that `signal(SIGCHLD, SIG_IGN)` line. Normally when a child exits it becomes a *zombie* — a husk the kernel keeps around so the parent can read its exit status with `wait`. A long-running server that never reaps its children would slowly fill the process table with zombies. Telling the kernel we want to *ignore* `SIGCHLD` is a shortcut that says "I don't care about exit codes, clean children up automatically."

The two `close` calls are the same discipline we learned with pipes: each side closes the fd it isn't using. The child closes the listener `s` (it'll never accept anyone); the parent closes `conn` (the child is handling that conversation). If the parent forgot to close `conn`, the kernel would still see an open copy of the fd even after the child closed its side, and the client would never get its EOF.

One thread per client: more efficient, but shared

`echoServerThread.c` does the same thing with a thread instead of a process. Threads are cheaper to spawn and share the process's memory — which is exactly the good news and the bad news. The good news is there's only one fd table, so we don't have to close anything in the "parent." The bad news is that all that shared mutable state is the race-condition minefield we spent two whole lectures on.

```
struct client {
  int conn;
  char ip[INET_ADDRSTRLEN];
  int port;
};

void* handle(void* arg){
  struct client* c = arg;
  pthread_detach(pthread_self()); // self-cleanup the moment we return

  char buf[1024];
  ssize_t n;
  while((n = recv(c->conn, buf, sizeof(buf), 0)) > 0){
    send(c->conn, buf, n, 0);
  }
  close(c->conn);
  free(c); // we malloc'd it, we free it
  return NULL;
}

// in main's accept loop:
struct client* c = malloc(sizeof(*c));
c->conn = conn;
c->port = ntohs(caddr.sin_port);
inet_ntop(AF_INET, &caddr.sin_addr, c->ip, sizeof(c->ip));

pthread_t tid;
pthread_create(&tid, NULL, handle, c);
```

```
// NOTE: unlike fork(), we do NOT close(conn) here -- there's no
// separate copy of the fd; the thread shares this exact one.
```

Two details worth dwelling on:

- **Why `malloc` a fresh `struct client` every loop?** If we passed the thread a pointer to a local variable, the next iteration of the accept loop would overwrite it out from under the running thread. Each thread needs its own heap-allocated box that lives until *it* is done, which is why the thread frees it at the end rather than the main loop.
- `pthread_detach` tells the runtime "nobody is going to `pthread_join` me, so reclaim my resources the instant I return." It's the threading analogue of the `SIGCHLD` trick — a way to fire-and-forget without leaking.

A tiny web server

Here's the fun payoff: HTTP is just text over a TCP socket, so our echo server is most of the way to a web server already. `webServer.c` accepts a connection, reads the request (which we mostly ignore), and writes back a hard-coded HTTP response.

```
while(1){
  int conn = accept(s, NULL, NULL);
  if(conn < 0) continue;

  char req[4096];
  ssize_t n = recv(conn, req, sizeof(req) - 1, 0);
  if(n <= 0){ close(conn); continue; }
  req[n] = 0;
  printf("--- request ---\n%s\n", req); // see: HTTP is just text

  char* body = "<html><body><h1>Hello from C!</h1>"
               "<p>this page came out of a 60-line server</p>"
               "</body></html>";
  char resp[1024];
  int rlen = snprintf(resp, sizeof(resp),
    "HTTP/1.1 200 OK\r\n"
    "Content-Type: text/html\r\n"
    "Content-Length: %zu\r\n"
    "Connection: close\r\n"
    "\r\n"
    "%s",
    strlen(body), body);

  send(conn, resp, rlen, 0);
  close(conn);
}
```

The shape of an HTTP response is worth memorizing because it's shockingly simple:

- A status line: `HTTP/1.1 200 OK`.
- Some headers, one per line, `Name: value`.

- A *blank line* that separates the headers from the body — this is mandatory and the single most common thing people get wrong.
- The body.

Note the line endings are `\r\n` (carriage-return + newline), not just `\n` — HTTP insists on it. And `Content-Length` has to match the body's actual byte count or the client gets confused about where the response ends. You can hit this three ways: `curl -v http://localhost:8080/`, by opening the URL in a real browser, or with our own `webClient.c`, which hand-rolls the request side:

```
char req[512];
int n = snprintf(req, sizeof(req),
  "GET %s HTTP/1.1\r\n"
  "Host: %s\r\n"
  "Connection: close\r\n"
  "\r\n",
  path, host);
send(s, req, n, 0);

char buf[4096];
ssize_t got;
while((got = recv(s, buf, sizeof(buf) - 1, 0)) > 0){
  buf[got] = 0;
  fputs(buf, stdout); // print the raw response, headers and all
}
```

A GET request is symmetric to the response: a request line (`GET /path HTTP/1.1`), headers, and the blank line that ends them. The `Host` header is required in HTTP/1.1 because one server might host many sites on one IP. This is exactly what your browser sends; ours just prints the whole raw reply, headers and all, so you can see there's no magic.

(Stretch) Handling many clients without fork or threads: `select` and `poll`

There's a third way to serve many clients that uses neither processes nor threads: stay single-threaded, but never block on any one client. Instead you hand the kernel your whole set of file descriptors and ask "which of these is ready to read right now?" This is the foundation of high-performance servers like nginx.

There are three generations of this idea:

- `select` — dates to early Unix, works everywhere, but is clumsy and has a hard limit on fd numbers.
- `poll` — a bit newer, no fd-number ceiling, nicer API.
- `epoll` — the modern, scales-to-many-thousands Linux version. We didn't write one, but it's where you'd go next.

An echo server with `select`

`select` works on an `fd_set`, a bitmask of file descriptors. The awkward part is that `select` *clobbers* the set it's handed — on return, the set only contains the fds that are ready — so you keep a master copy (`reference`) and copy it into a scratch set each time through the loop.

```
fd_set reference, readfds;
FD_ZERO(&reference);
FD_SET(s, &reference); // start by watching just the listener
int maxfd = s; // select wants the highest fd, plus one

while(1){
  readfds = reference; // fresh copy; select will chew on this one
  select(maxfd + 1, &readfds, NULL, NULL, NULL);

  for(int fd = 0; fd <= maxfd; fd++){
    if(!FD_ISSET(fd, &readfds)) continue; // not ready, skip

    if(fd == s){
      // the listener is readable => a new client is waiting
      int conn = accept(s, (struct sockaddr*)&caddr, &clen);
      FD_SET(conn, &reference); // start watching them
      if(conn > maxfd) maxfd = conn;
    } else {
      // an existing client has data (or hung up)
      char buf[1024];
      ssize_t n = recv(fd, buf, sizeof(buf), 0);
      if(n <= 0){
        close(fd);
        FD_CLR(fd, &reference); // stop watching a dead fd
      } else {
        send(fd, buf, n, 0);
      }
    }
  }
}
```

The trick that makes the whole thing work is that the *listening socket* is just another fd in the set. When it becomes "readable," that doesn't mean there are bytes to `recv` — it means a client is waiting to be `accept`ed. Every other ready fd is a real client with data. One loop, no forking, no threads, and one program juggling everyone. The weird bits are real: you have to track `maxfd` by hand, re-copy the set every iteration, and scan every fd from `0` to `maxfd` whether it's in the set or not.